Limits on the Power of Constrained PRFs and Identity-based Cryptography

Roman Langrehr[⋆]

University of Waterloo, Canada
roman.langrehr@uwaterloo.ca

Abstract. Constrained PRFs are PRFs that allow the generation of constrained keys, which can be used to evaluate the PRF on a subset of the inputs. The PRF is still pseudorandom for an adversary who obtains multiple of these constrained keys on all inputs where none of the constrained keys allow it to evaluate the PRF. Constrained PRFs are known for some simple constraint classes (such as puncturing or intervals) from one-way functions, but for more powerful constraint classes, such as bitfixing, the only known constructions need heavy machinery, like indistinguishability obfuscation or multilinear maps.
In this work we show that constrained PRFs (for any constraint class) do not imply key agreement in a black-box way with an oracle separation. The result also applies to delegatable constrained PRFs, where constrained keys can be used to generate other, more restrictive constrained keys.
We show that this result has interesting implications for identity-based cryptography, where users obtain secret keys from a trusted, central authority. Namely, it shows that primitives that allow key agreement in this setting, like identity-based non-interactive key exchange and a weaker variant of identity-based encryption that we introduce in this work, do not imply key agreement and thus can exist even in a world where traditional key agreement and public-key encryption is impossible.

1 Introduction

1.1 Constraint PRFs

A pseudorandom function (PRF) is a function $f : \mathcal{K}_\lambda \times \mathcal{X}_\lambda \to \mathcal{Y}_\lambda$ such that a PPT adversary with oracle access to the function $f(K, \cdot)$ for a uniform random key $K \xleftarrow{\$} \mathcal{K}_\lambda$ cannot distinguish this function from a random function $f' : \mathcal{X}_\lambda \to \mathcal{Y}_\lambda$. A constrained PRF for a class of constraints $\mathcal{C}_\lambda \subseteq \mathfrak{P}(\mathcal{X}_\lambda)$ is a PRF with the following extra functionality: For each constraint set $C \in \mathcal{C}_\lambda$ we can derive a constrained key $K\langle\!\langle C \rangle\!\rangle$ that allows to evaluate $f(K, x)$ for $x \in \mathcal{X}_\lambda$ but $f(K, x)$ should still be pseudorandom to an adversary that has only obtained CPRF keys that do not allow evaluation of the PRF on x. Unless stated otherwise, we assume that this holds even for adversaries who can request as many constrained keys as they want (unbounded collusion-resistance).

[⋆] Part of the work was done while the author was affiliated with ETH Zurich.

The concept of CPRFs was introduced independently in [14, 16, 50] and all of them noted that the GGM PRF [40] can be used as a constrained PRF for prefix constraints, where $\mathcal{X}_\lambda$ is a bitstring and each constraint set $C \in \mathcal{C}_\lambda$ consists of all bitstrings with a fixed common prefix. It is easy to tune this construction to other simple constraint classes, such as intervals. Boneh and Waters [14] also gave a construction of a left/right CPRF (L/R CPRF), where each constraint sets $C \in \mathcal{C}_\lambda$ either consists of all bitstrings whose left half is a fixed bitstring or whose right half is a fixed bitstring. Their construction uses parings and the random oracle model and is similar to the SOK ID-NIKE [65] and the Boneh-Franklin IBE [9]. They also gave constructions for bitfixing CPRFs (where each constraint sets $C \in \mathcal{C}_\lambda$ consists of all bitstrings that match a pattern that fixes some of the bits) and, with a more involved construction, a CPRF where the constraint sets are described by arbitrary circuits using mutlilinear maps. CPRFs for circuits are also known from FE [27]. Even more powerful are CPRFs for Turing machines, where first constructed based on differing-input obfuscation [1] and later using only indistinguishability obfuscation (for circuits) [32, 30].

More constructions of CPRFs are known in the weaker bound-collusion model, where the adversary can only obtain a fixed number of constrained keys [19, 2, 61, 31, 26, 67].

Adaptive security for constrained PRFs is very challenging to achieve. Even for the basic GGM construction, this was only achieved recently in a beautiful work by Hofheinz, Kastner, and Klein [46] (improving upon a reduction with quasipolynomial loss [38] and circumventing a prior impossibility result [49]). For more powerful constraint classes, adaptively secure constructions are only known from universal samplers [45], which in turn are only known from indistinguishability obfuscation (iO) in the ROM [44]. For bit-fixing CPRFs, Fuchsbauer et al. [38] showed that simple reductions to non-interactive assumptions have to incur an exponential loss. Recently, Cheng and Jaeger [25] introduced a stronger, simulation-based definition of adaptive security for CPRFs, but this notion is only achievable in idealized models.

Many works have also studied additional properties for constrained PRFs, such as delegatability (where constrained keys can be constrained further) [14, 50, 23, 28, 11], verfiability [37, 23, 28, 20], key homomorphic CPRFs [3, 19], private constrained PRFs (where constrained keys are guaranteed to hide their constraint sets) [12, 10, 21, 18, 60], and invertible CPRFs [11, 60].

Constrained PRFs are a versatile tool that is used in countless cryptographic constructions. Especially in combination with iO, they imply a plethora of powerful cryptographic primitives [64].

1.2 The power of constrained PRFs

Despite these various applications of constrained PRFs, very little is known about what CPRFS imply alone, because most applications of CPRFs use them in combination with other primitives. Notable exceptions are a result by Boneh and Waters [14] showing that left/right constrained PRFS imply identity-based non-interactive key exchange (ID-NIKE) and a result by Hofheinz [43], that

shows that bitfixing CPRFs imply multiparty ID-NIKE.[1] However, this does not give us any meaningful insights about what assumptions are needed for CPRFs, because for (multi-party) ID-NIKE we also know nothing non-trivial about the minimal assumptions. Essentially, the state of the art for these primitives is that they can be constructed from iO and OWFs and that they imply OWFs, which leaves an enormous gap what the minimal assumptions are.

A notable barrier is a result by [21], which shows that private CPRFs for all circuits imply iO, even in the 2-key setting, where the adversary can obtain at most two constrained keys. However, this result does not imply anything for simpler constraint classes, like L/R CPRFs and bitfixing CPRFs, and nothing about non-private CPRFs.

The main result of this work is an oracle separation showing that key agreement cannot be constructed from CPRFs in a black-box way.

Informal Theorem 1. There exists no black-box construction of a perfectly correct key agreement scheme from constrained PRFS for any constraint class.

The result applies to all constraint classes and also to delegatable CPRFs. Our result even rules out that CPRFs with adaptive[2] security implies key agreement. Our result only rules out perfectly correct key agreement and this limitation is inherent to the technique this work is based on. We refer the reader to the technical overview for more details and possible approaches to overcome this limitation.

Open problems. The vast majority of CPRF constructions uses assumptions that imply key agreement (and are often much more powerful than that, like multilinear maps and iO), with the notable exception of constructions based on the GGM PRF. Our result shows that these assumptions are not minimal and we think it is an exciting direction for future research to further narrow down what the minimal assumptions for CPRFs are, especially for simple constraint classes such as L/R CPRFs and bitfixing CPRFs. In particular, constructions from concrete (algebraic) assumptions that are weaker or incomparable to key agreement would be very exciting. Our result shows that for any CPRF any assumptions that implies key agreement is *not* minimal. We conjecture that it is not possible to construct left/right CPRFs (and thus CPRFs for any more powerful constraint class) from one-way functions alone.

A concurrent work to this one proves that 1-key private CPRFs also do not imply key agreement in a black-box way [6]. Their result is incomparable to ours. On the one hand, it is stronger because their separation applies even to private CPRFs. On the other hand, our result is stronger because it applies to unbounded-collusion secure and to delegatable CPRFs.

[1] The paper [43] has been merged into [45], but the multiparty ID-NIKE construction is only included in the original work by Hofheinz.

[2] In this work we use the standard, indistinguishability based variant of adaptive security. Our result can be easily adapted to CPRFs achieving the simulation-based definition of [25] in the programmable oracle model. In the standard model or non-programmable oracle model, their notion is impossible to achieve.

1.3 Identity-based cryptography

Our result has surprising consequences for identity-based cryptography, a concept introduced by Shamir [68]. In identity-based cryptography users are associated with an identity. This identity replaces the role of the public key in public-key cryptography. In contrast to a public key, the identity can carry meaningful information about the owner of the identity (e.g. it can be their e-mail address, website domain, or orcid number) as long as identities are unique. Each user then obtains a user secret key for their identity from a trusted party, that also publishes a master public key (for some identity-based primitives). Many works also consider a hierarchical variant, where we have a hierarchy of key generators.

Popular identity-based primitives are identity-based signatures (which are known to be equivalent to regular signatures [68]), identity-based encryption and identity-based non-interactive key exchange. The latter two are discussed below in more detail.

ID-NIKE. Identity-based non-interactive key exchange (ID-NIKE) enables users to derive a common shared key without interaction. More concretely, each user (say, Alice) can use their own user secret key and the identity of someone else (say, Bob) to derive a shared key. If Bob does the same with his user secret key and Alice's identity, he gets the same shared key as Alice. This can also be generalized to multiparty ID-NIKE, where a shared key for a group of users can be obtained by every member of the group.

Identity-based NIKE has a long history. The notion was first formalized in [34, 59], but the idea was studied even before these works. A work by Blom [7] proposed a construction that can be seen as a bounded-collusion secure ID-NIKE already in 1984. Blundo et al. [8] generalized this to multiparty and hierarchical ID-NIKE and several follow-up works studied variants of this scheme [35, 39, 62]. Unbounded-collusion ID-NIKE was first studied by Maurer and Yacobi [55, 53] in the early 1990's.These constructions all built upon trapdoor discrete logarithm (TD-DL) groups. Unfortunately, the original proposals for such groups where all subsequently broken [54, 51, 57].We refer the reader to [59] for an excellent overview of this approach. They also give two instantiations of these groups, where in the resulting ID-NIKE the key generator is inefficient (but it is more efficient than brute-force searching for a user secret key).

The first full-fledged ID-NIKE was presented by Sakai, Ohgishi, and Kasahara [65] and studied in [34, 59, 24]. It uses pairings and the random oracle model and is similar to the Boneh-Franklin IBE [9]. Boneh and Zhandry [15] presented an ID-NIKE from iO and OWFs and Freire, Hofheinz, Paterson, and Striecks [36] presented an ID-NIKE from multilinear maps.

All of these ID-NIKE constructions require very powerful assumptions. However, Boneh and Waters [14] showed that ID-NIKE can be constructed from L/R CPRFs and Hofheinz [43] showed that bitfixing CPRFs imply multiparty NIKE. In this work, we show that ID-NIKE is in fact equivalent to L/R CPRFs and N-party ID-NIKE is equivalent to "1-out-of-N" block-fixing CPRFs, a slightly weaker

variant than bitfixing CPRFs. Moreover, we show that hierarchical (multiparty) ID-NIKE (HID-NIKE) can be built from delegatable block-fixing CPRFs.

Thus, a consequence of our main result Informal Theorem 1 is that ID-NIKE (and variants of it like multiparty or hierarchical ID-NIKE) do not imply traditional key agreement in a black-box way. Consequently, key agreement in the identity-based setting is weaker or incomparable to traditional key agreement.

Paterson and Srinivasan [59] showed that ID-NIKE implies IBE assuming a certain structure of the ID-NIKE. These properties are satisfied by the TD-DL group based ID-NIKEs and the SOK ID-NIKE (in the latter case the resulting IBE is the Boneh-Franklin IBE [9]), but not by the standard model ID-NIKE schems from iO or multilinear maps. Our result implies that this implication does not hold for general ID-NIKE (for black-box constructions).

A recent work [22] has introduced multi-authority ID-NIKE, a more powerful variant of ID-NIKE, which trivially implies key agreement.

Gated IBE. Identity-based encryption (IBE) allows a sender to encrypt a message using only the master public key and the identity of the recipient. IBE is a very well-studied primitive. It trivially implies PKE (and therefore key agreement), but is actually harder to construct than PKE. For example, it is known to be impossible in the generic group model [58, 66, 70] or from trapdoor permutations [13] in a black-box way. The impossibility of generic groups has been circumvented by Döttling and Garg [33] in a celebrated work.

In this work we introduce a novel relaxation of IBE which we call gated IBE. In gated IBE, the encryptor needs to obtain a user secret key to be able to encrypt messages. This is in contrast to regular IBE, where only the master public key is need for encryption. Security of this scheme is essentially guaranteed as long as neither the sender nor the receiver's user secret key have been compromised. Since the sender and receiver both know the plaintext of an encrypted ciphertext anyway, it is a reasonable model that neither of them collaborates with the adversary and the adversary does not obtain their secret keys.

This IBE variant can be used in similar applications as traditional IBE. Consider for example a messaging or e-mail software that is used in a company or other institution for their internal communication. The head of the company can serve as key generator and generate a user secret key for every employee. Employees can then encrypt their messages with a gated IBE to protect them from an outsider and from other employees, apart from the head of the company.

We show that gated IBE (even an IND-CCA secure variant) can be built from ID-NIKE. We also show that IBE (trivially) implies gated IBE. Both results translate to the hierarchical setting.

Thus, another consequence of our main result (Informal Theorem 1) is that gated IBE does not imply key agreement (and therefore PKE and IBE) in a black-box sense.

If we relax the notion of IBE further and allow encryption only using the master secret key (sk-IBE), this notion is trivally implied by OWFs as noted in [42]. We conjecture that gated IBE cannot be built from OWFs and therefore lies strictly in between sk-IBE and regular IBE.

Single-authority identity-based cryptography. The crucial property of gated IBE and ID-NIKE, that enables the separation from key agreement, is that they provide their functionality (non-interactive key exchange or encryption) only to users who have obtained their user secret keys from the same key authority. We call such schemes single-authority identity-based schemes.

In contrast, multi-authority identity-based schemes such as regular IBE and multi-authority ID-NIKE [22] provide their functionality to users that have obtained their secret keys from different key authorities or do not have obtained a user secret key at all.

We give an overview of the primitives discussed here in Figure 1.

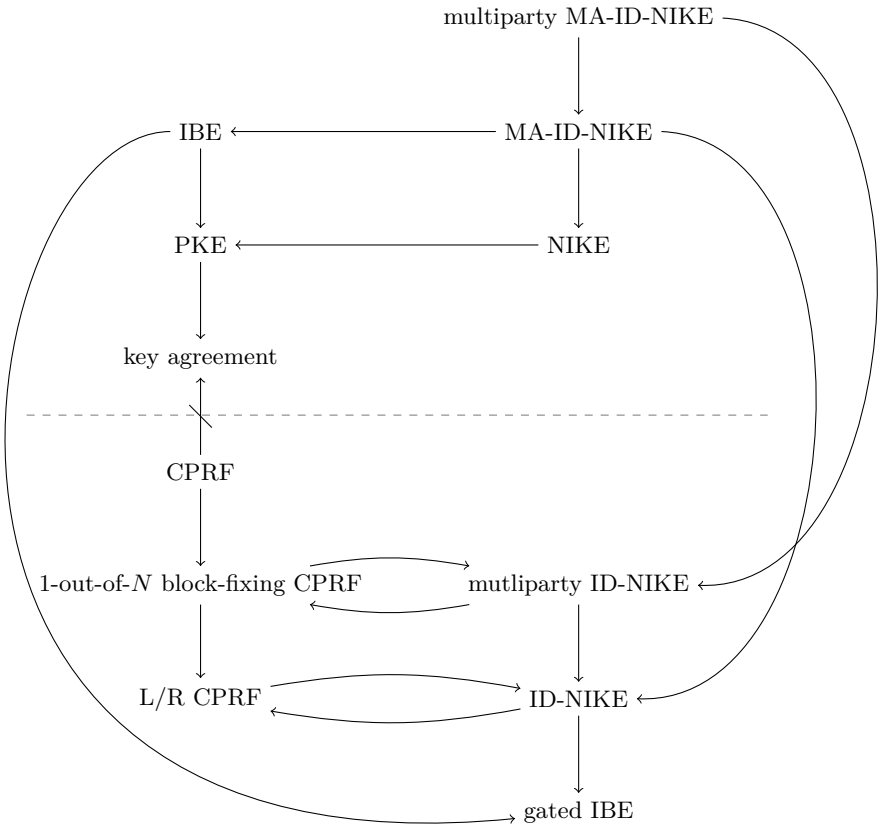

Fig. 1. The relation between the primitives discussed in this work. Similar results hold for delegatable CPRFs/hierarchical identity-based schemes, but are omitted here for clarity. Our result separates all primitives below the dashed line from all primitives above the dashed line.

1.4 Organization

We give a technical overview of the proof of Informal Theorem 1 and compare it with previous oracle separations for key agreement in Section 2.

In Section 3 we introduce notation and the formal definitions of delegatable CPRFs, key agreement and commitments. Our definition of delegatable CPRFs deviates from prior work, and we discuss the differences and why previous definitions were unsuitable for our applications.

In Section 4 we present our main result, the oracle separation of key agreement from constrained PRFs. In Appendix A, we present the formal definition of ID-NIKE. In Appendix B we show that ID-NIKE implies L/R CPRFs, which complements [14] to show the equivalence of ID-NIKE and L/R CPRFs. In Appendix C, we show that N-party ID-NIKE is equivalent to one-out-of-N block-fixing CPRFs. The direction that one-out-of-N block-fixing CPRFs imply N-party ID-NIKE is an adaption of a result by Hofheinz [43] that uses bit-fixing CPRFs instead. In Appendix D we give a formal definition of HID-NIKE (similar notions have been studied in [8, 39] and in [22] in the multi-authority setting) and show how to construct it from delegatable block-fixing CPRFs. Finally, in Appendix E we present a formal definition for gated IBE and HIBE, our new (H)IBE variants, and show that they can be built from (H)ID-NIKE or IBE.

2 Technical Overview

2.1 Oracle separations.

We cannot hope to prove a statement like "cryptographic primitive X implies cryptographic primitive Y" in the propositional logic sense of "imply", because Y might exist unconditionally. An oracle separation shows that there exists an oracle $\mathcal{O}$, such that X exists relative to $\mathcal{O}$ but Y does not. Such a result shows that there is no construction of Y that uses X only in a *black-box* way [63]. Since the vast majority of cryptographic constructions and techniques are black-box (especially when we consider only "efficient" constructions), such a result is a meaningful barrier of constructing X from Y.

Typically, the oracle $\mathcal{O}$ consists of a suboracle that implements an idealized functionality of X, to ensure that X exists, and a **PSPACE** suboracle, to ensure that no cryptographic primitves other than X exists. In this work, we omit the **PSPACE** suboracle and consider instead security against unbounded adversaries that make only polynomially many $\mathcal{O}$ queries, which has a very similar effect to the **PSPACE** suboracle.[3]

2.2 Impagliazzo and Rudich's result.

The technique of oracle separations was pioneered by Impagliazzo and Rudich [48], who proved that there is no key agreement scheme relative to a random

[3] An advantage of this approach is that we do not have to require that constraint sets have an efficient representation, which would be needed for PPT adversaries with a **PSPACE** oracle.

oracle. This rules out black-box constructions of key agreement from a plethora of cryptographic primitives such as one-way functions, one-way permutations, pseudorandom generators and functions, secret key encryption, digital signatures, and collision-resistant hash functions.

Informally speaking, a key agreement is a protocol between two users (Alice and Bob) that exchange messages and output in the end a shared key that agrees with probability $1 - \varepsilon_{\mathsf{corr}}$. Security guarantees that an adversary Eve who gets a transcript of all exchanged messages should not be able to guess the key that Bob outputs. For many applications, a security notion where Eve should not be able to distinguish Bob's key from a uniformly random key is often more useful, but for this work we use the variant where Eve should not be able to predict the shared key.

On a high level, Impagliazzo and Rudich [48] consider an adversary (Eve) that proceeds in several segments for each round of the key agreement scheme. In each segment Eve simulates the party who speaks in that round (say, Alice) using *simulated* oracle answers and a simulated randomness tape conditioned on being consistent with all messages Alice has sent so far and all query-answer pairs of the oracle $\mathcal{O}$ that Eve has learned so far. Afterward, Eve queries all oracle queries that Alice has made in the simulated run to the *real* oracle.

The main work of their proof is to show (via a rather tedious argument) that when Eve makes enough of these segments per round, Eve learns with high probability all oracle queries that where done both by the *real* Alice and the *real* Bob. Without loss of generality Alice and Bob query the random oracle on their shared key in the last round. Thus, Eve can output one of the queries she learned and with noticeable probability this will be the correct query.

Their proof was subsequently improved by Barak and Mahmoody [4], who essentially optimized Eve to make only $\mathcal{O}(q^2)$ queries, where q is the number of queries Alice and Bob make, thus showing the optimality of Merkle puzzles [56].

A simpler proof. Brakerski, Katz, Segev, and Yerukhimovich [17] give a much simpler proof of this classic result. However, their proof only rules out perfectly correct key agreement. This proof will be the starting point to prove our main result.

First, we recall the proof by [17]. They consider an adversary Eve that proceeds in $2q_B + 1$ rounds, where q_B is the number of queries made by Bob. Each round consists of a simulation phase and an update phase. In the simulation phase, Eve simulates a run of Alice that is consistent with Eve's current view of the protocol execution. In the first round, Eve's view consists only of the transcript, so Eve samples a random tape and oracle answers for Alice such that all messages Alice sends match the corresponding messages of the real transcript. When Alice is expecting a message from Bob, Eve feeds Alice the message that Bob send in the real transcript.

In the update phase, Eve queries the random oracle on all queries Alice has made during the simulation phase. For subsequent rounds, Eve will answer in the simulation phase these queries with the real value, and only provide a random response for unknown queries.

In the end, Eve outputs the majority value of the shared key that the simulated Alice outputs in each round.

We call rounds "bad" if Eve had to simulate a query for Alice with a random answer that Bob also made during the real run. We call this an intersection query. There are at most q_B bad rounds, because in each bad round Eve learns the real answer for intersection queries in the update phase.

In good rounds, the queries by the real Bob and the simulated Alice are answered consistently. By perfect correctness, the simulated Alice outputs in these rounds the same shared key as the real Bob. Since there are at least $q_B + 1$ good rounds, this shared key will be the majority key.

2.3 Separating CPRFs from key agreement.

Next, we outline the main challenges to extend the above result to constrained PRFs. First of all, we need a more powerful oracle such that CPRFs exist relative to this oracle. A natural candidate for such an oracle is the following oracle that consists of two suboracles $\mathcal{O}_{\mathsf{eval}}$ and $\mathcal{O}_{\mathsf{constrain}}$ defined as follows.

- $\mathcal{O}_{\mathsf{constrain}} : \mathcal{K}_\lambda \times \mathcal{C}_\lambda \to \mathcal{CK}_\lambda$ is a random permutation
- $\mathcal{O}_{\mathsf{eval}} : (\mathcal{K}_\lambda \cup \mathcal{CK}_\lambda) \times \mathcal{X}_\lambda \to \mathcal{Y}_\lambda \cup \{\bot\}$ where
  - $\mathcal{O}_{\mathsf{eval}}\big|_{\mathcal{K}_\lambda} : \mathcal{K}_\lambda \times \mathcal{X}_\lambda \to \mathcal{Y}_\lambda$ is a random function
  - $\mathcal{O}_{\mathsf{eval}}(\overline{K}, x) = \mathcal{O}_{\mathsf{eval}}(K, x)$ if $\mathcal{O}^{-1}_{\mathsf{constrain}}(\overline{K}) = (K, C)$ with $x \in C$
  - $\mathcal{O}_{\mathsf{eval}}(\overline{K}, x) = \bot$ if $\mathcal{O}^{-1}_{\mathsf{constrain}}(\overline{K}) = (K, C)$ with $x \notin C$

Relative to this oracle, a CPRF with key space $\mathcal{K}_\lambda$, constraint class $\mathcal{C}_\lambda$, input space $\mathcal{X}_\lambda$ and output space $\mathcal{Y}_\lambda$ trivially exists: KeyGen just returns a uniformly random key $K \xleftarrow{\$} \mathcal{K}_\lambda$ and $\mathsf{Constrain}$ and Eval just call the $\mathcal{O}_{\mathsf{constrain}}$ and $\mathcal{O}_{\mathsf{eval}}$ on their respective inputs. Note that this CPRF is not constraint hiding, because the $\mathcal{O}_{\mathsf{eval}}$ oracle can be used to test whether an element is in the constraint set of a constrained key or not.

The oracle above does not allow us to build a delegatable CPRF, but this can be modeled by accepting constrained keys in $\mathcal{O}_{\mathsf{constrain}}$, too (as long as they are constrained on a superset of the new constrained key). We omit this for the purpose of this overview, because the difficulties this causes are similar to the difficulties we encounter with the $\mathcal{O}_{\mathsf{eval}}$ oracle above, which already accepts both unconstrained and constrained keys.

The difficulty of handling constrained PRFs. The problem with this oracle is that there are complex correlations between the answers of the oracles, which an adversary following the proof strategy from above has to take into account. However, for this oracle this seems to be impossible. Consider for example the following scenarios:

Scenario 1: Fix two constraints $C_0, C_1 \in \mathcal{C}_\lambda$ and $x \in \mathcal{X}_\lambda$ with $x \in C_0$ and $x \notin C_1$. Bob samples $K \xleftarrow{\$} \mathcal{K}_\lambda$, a secret bit b, $\overline{K} \leftarrow \mathcal{O}_{\mathsf{constrain}}(K, C_b)$ and sets $\overline{K}$ to Alice. Alice queries $\mathcal{O}_{\mathsf{eval}}(\overline{K}, x)$.

Scenario 2: Bob samples two CPRF keys $K_0, K_1 \xleftarrow{\$} \mathcal{K}_\lambda$ and a secret bit b. Bob then sends K_0 and $\overline{K} := \mathcal{O}_{\mathsf{constrain}}(K_b, C)$ to Alice. Alice queries $\mathcal{O}_{\mathsf{eval}}(K_0, x)$ and $\mathcal{O}_{\mathsf{eval}}(\overline{K}, x)$ on some value $x \in C$.

In both scenarios, Bob makes only a $\mathcal{O}_{\mathsf{constrain}}$ query and Alice makes only $\mathcal{O}_{\mathsf{eval}}$ queries. Thus, the statement that Eve has learned all the intersection queries between the simulated Alice and the real Bob is trivially true in both of these scenarios, no matter what query-answer pairs Eve has actually learned.

Despite this, it is actually difficult for Eve to faithfully simulate the oracle answers for Alice (such that they are consistent with Bob's random tape). In the first scenario, the answer to Alice's $\mathcal{O}_{\mathsf{eval}}(\overline{K}, x)$ query is $\bot$ or a uniform random value from $\mathcal{Y}_\lambda$, depending on Bob's secret bit b. In the second scenario the answers to Alice's two $\mathcal{O}_{\mathsf{eval}}$ queries are the same uniform random value from $\mathcal{Y}_\lambda$, if $b = 0$, and independent uniform random value from $\mathcal{Y}_\lambda$, otherwise. Thus, in both scenarios simulating the queries faithfully requires knowledge of a secret that cannot be inferred from the intersection of Alice's and Bob's queries.

Non-solution: Closures. A technique to handle correlated oracle queries is to define a suitable *closure* $\widetilde{\mathsf{QA}}$ of a set of query-answer pairs QA [41]. The idea is that this closure $\widetilde{\mathsf{QA}}$ contains all queries that can be inferred from the queries in QA. Instead of arguing that Eve has learned all the intersection queries between simulated Alice and real Bob, we would then argue that she learns the intersection queries of the *closures* of their queries.

Unfortunately, this technique cannot solve the problems we encounter in the two scenarios above, for different reasons. We write in the following $\mathsf{QA}_{\mathsf{eval}}$ and $\mathsf{QA}_{\mathsf{constrain}}$ for the query answer pairs in QA corresponding to the suboracles $\mathcal{O}_{\mathsf{eval}}$ and $\mathcal{O}_{\mathsf{constrain}}$, and, similarly, $\widetilde{\mathsf{QA}}_{\mathsf{eval}}$ and $\widetilde{\mathsf{QA}}_{\mathsf{constrain}}$ for the query answer pairs in $\widetilde{\mathsf{QA}}$ corresponding to the suboracles $\mathcal{O}_{\mathsf{eval}}$ and $\mathcal{O}_{\mathsf{constrain}}$.

To resolve the problem in scenario 1 with the closure technique, we would have to add for every $((K, C), \overline{K})$ query-answer pair in $\mathsf{QA}_{\mathsf{constrain}}$ all query-answer pairs of the form $((\overline{K}, x), \bot)$ with $x \notin C$ to $\mathsf{QA}_{\mathsf{eval}}$. This solves the problem we mentioned above for scenario 1, but it creates a new problem: The closure $\widetilde{\mathsf{QA}}$ can be exponentially larger than QA. With the proof strategy described above, this would force Eve to make an exponential number of rounds and thus exponentially many real $\mathcal{O}$ queries.

For scenario 2, we cannot even solve the problem if we define $\widetilde{\mathsf{QA}}$ to be all query-answer pairs that are determined by QA. The reason is that we can infer the answer for an $\mathcal{O}_{\mathsf{eval}}(\overline{K}, x)$ query only when we know both $((K, C), \overline{K}) \in \mathsf{QA}_{\mathsf{constrain}}$ and $((\overline{K}, x), y) \in \mathsf{QA}_{\mathsf{eval}}$. In the above scenario Bob makes $\mathcal{O}_{\mathsf{constrain}}(K, C) = \overline{K}$ query and Alice makes the $\mathcal{O}_{\mathsf{eval}}(\overline{K}, x) = y$ query. Thus, the answer to the $\mathcal{O}_{\mathsf{eval}}(\overline{K}, x)$ is in the closure of the *union* of Alice's and Bob's queries, but neither in the closure of Alice's query-answer pairs nor in the closure of Bob's query-answer pairs. Therefore, Eve cannot infer the correct answer for this query even if she would know all intersection queries in the closures of Alice's and Bob's queries.

Our solution. We solve the issue that arises in scenario 1 by adding an oracle $\mathcal{O}_{\mathsf{cset}}$ that inputs a constrained key $\overline{K}$ and outputs the constraint set C that was used to generate $\overline{K}$. With this oracle, we can enforce that each algorithm only makes $\mathcal{O}_{\mathsf{eval}}(\overline{K}, x)$ queries when $x \in \mathcal{O}_{\mathsf{cset}}(\overline{K})$, by querying $\mathcal{O}_{\mathsf{cset}}(\overline{K})$ before each $\mathcal{O}_{\mathsf{eval}}(\overline{K})$ query with a constrained key $\overline{K}$. Of course, this causes a new problem: The answers to $\mathcal{O}_{\mathsf{cset}}$ are correlated with the answers of the $\mathcal{O}_{\mathsf{constrain}}$ oracle. However, this can be dealt with using the closure technique described above.

Our solution to the problem that arises in scenario 2 is more involved, because we cannot reveal the PRF key K that was used to generate a constrained key $\overline{K}$, because this would break the CPRF security. Instead, we associate each PRF key K with a label L. This label can be obtained via a new oracle $\mathcal{O}_{\mathsf{label}}$. Moreover, we add an oracle $\mathcal{O}_{\mathsf{clabel}}$ that inputs a constrained key $\overline{K}$ and outputs the label of the PRF key that was used to generate $\overline{K}$. Since this label cannot be used to evaluate the CPRF, this does not break the CPRF security. By forcing each algorithm to query $\mathcal{O}_{\mathsf{label}}$ or $\mathcal{O}_{\mathsf{clabel}}$ before using a (unconstrained or constrained) key in the $\mathcal{O}_{\mathsf{eval}}$ oracle, we can ensure that when Eve has learned all the intersection queries between (closures of) Alice's and Bob's queries, she can correctly simulate Alice's queries. Again, we have to make sure that $\mathcal{O}_{\mathsf{clabel}}$ queries are answered consistently with $\mathcal{O}_{\mathsf{constrain}}$ queries, but this can be done with the closure technique.

To enforce the querying behavior we described above, and to simplify the description of how Eve answers queries, we introduce two internal suboracles $\mathcal{O}_{\mathsf{ieval}}$ and $\mathcal{O}_{\mathsf{iconstrain}}$. These oracles are not accessible directly and cannot be called by Alice, Bob, or Eve. They can only be called by other suboracles. These suboracles perform essentially what $\mathcal{O}_{\mathsf{eval}}$ and $\mathcal{O}_{\mathsf{constrain}}$ did in the previous description, but they operate on the labels instead of the actual (constrained) PRF keys. These internal suboracles also make the description of how Eve answers queries consistently much simpler.

This leads us to the following oracle model with the following suboracles. For simplification, we have added the functionality of $\mathcal{O}_{\mathsf{cset}}$ to $\mathcal{O}_{\mathsf{clabel}}$. We have also added the constrained key delegation functionality.

1. The internal suboracle $\mathcal{O}_{\mathsf{ieval}} : \mathcal{L}_\lambda \times \mathcal{X}_\lambda \to \mathcal{Y}_\lambda$ is a random function.
2. The internal suboracle $\mathcal{O}_{\mathsf{iconstrain}} : \mathcal{L}_\lambda \times \mathcal{C}_\lambda \to \mathcal{CK}_\lambda$ is a random injective function.
3. The public suboracle $\mathcal{O}_{\mathsf{label}} : \mathcal{K}_\lambda \to \mathcal{L}_\lambda$ is a random injective function.
4. The public suboracle $\mathcal{O}_{\mathsf{clabel}} : \mathcal{CK}_\lambda \to (\mathcal{L}_\lambda \times \mathcal{C}_\lambda) \cup \{\bot\}$ is the inverse of $\mathcal{O}_{\mathsf{iconstrain}}$ in the following sense: $\mathcal{O}_{\mathsf{clabel}}(\overline{K}) = \mathcal{L}_\lambda \times \mathcal{C}_\lambda$ iff $\mathcal{O}_{\mathsf{iconstrain}}(\mathcal{L}_\lambda \times \mathcal{C}_\lambda) = \overline{K}$ and $\mathcal{O}_{\mathsf{clabel}}(\overline{K}) = \bot$ iff $\overline{K}$ is not in the image of $\mathcal{O}_{\mathsf{iconstrain}}$.
5. The public suboracle $\mathcal{O}_{\mathsf{eval}} : (\mathcal{K}_\lambda \cup \mathcal{CK}_\lambda) \times \mathcal{X}_\lambda \to \mathcal{Y}_\lambda \cup \{\bot\}$ is implemented as follows:
   $\underline{\mathcal{O}_{\mathsf{eval}}(K, x) \text{ with } K \in \mathcal{K}_\lambda:}$
   $L \leftarrow \mathcal{O}_{\mathsf{label}}(K)$
   return $\mathcal{O}_{\mathsf{ieval}}(L, x)$
   $\underline{\mathcal{O}_{\mathsf{eval}}(\overline{K}, x) \text{ with } \overline{K} \in \mathcal{CK}_\lambda}$
   $(L, C) \leftarrow \mathcal{O}_{\mathsf{clabel}}(\overline{K})$

if $x \in C$ **then return** $\mathcal{O}_{\mathsf{ieval}}(L, x)$ **else return** $\bot$

6. The public suboracle $\mathcal{O}_{\mathsf{constrain}} : (\mathcal{K}_\lambda \cup \mathcal{CK}_\lambda) \times \mathcal{C}_\lambda \to \mathcal{CK}_\lambda \cup \{\bot\}$ is implemented as follows:
   $\underline{\mathcal{O}_{\mathsf{constrain}}(K, C) \text{ with } K \in \mathcal{K}_\lambda\text{:}}$
   $L \leftarrow \mathcal{O}_{\mathsf{label}}(K)$
   return $\mathcal{O}_{\mathsf{iconstrain}}(L, C)$
   $\underline{\mathcal{O}_{\mathsf{constrain}}(\overline{K}, C') \text{ with } \overline{K} \in \mathcal{CK}_\lambda}$
   $(L, C) \leftarrow \mathcal{O}_{\mathsf{clabel}}(\overline{K})$
   if $(L, C) = \bot$ **then return** $\bot$
   else if $C' \subseteq C$ **then return** $\mathcal{O}_{\mathsf{iconstrain}}(L, C)$ **else return** $\bot$

The suboracles $\mathcal{O}_{\mathsf{eval}}$ and $\mathcal{O}_{\mathsf{constrain}}$ are efficiently implemented using the first four oracles. Therefore, when we apply the proof strategy outline above, we only keep track of the query-answer pairs of the first four oracles. Queries to $\mathcal{O}_{\mathsf{eval}}$ and $\mathcal{O}_{\mathsf{constrain}}$ are always answered running the implementation above and simulating or executing the internal oracles calls these oracles made. These oracles only play a role in the analysis of Eve's advantage, where we exploit that $\mathcal{O}_{\mathsf{ieval}}$ and $\mathcal{O}_{\mathsf{iconstrain}}$ can only be called via $\mathcal{O}_{\mathsf{eval}}$ and $\mathcal{O}_{\mathsf{constrain}}$ queries.

The oracles $\mathcal{O}_{\mathsf{iconstrain}}$ and $\mathcal{O}_{\mathsf{clabel}}$ are redundant, but neither of them can be efficiently implemented using the other one. Thus, we have to keep track of their query-answer pairs separately and use the closure technique to enforce that these oracles are consistent.

Eve proceeds as in the proof sketched above, but the update phase becomes more involved with this oracle. For $\mathcal{O}_{\mathsf{label}}$ and $\mathcal{O}_{\mathsf{clabel}}$ queries, Eve can directly query the real oracle. For $\mathcal{O}_{\mathsf{ieval}}$ and $\mathcal{O}_{\mathsf{iconstrain}}$, Eve looks up a suitable (constrained) key with the correct label in her query-answer pairs and makes a corresponding $\mathcal{O}_{\mathsf{eval}}$ or $\mathcal{O}_{\mathsf{constrain}}$ query. Since these oracles can only be accessed through $\mathcal{O}_{\mathsf{constrain}}$ or $\mathcal{O}_{\mathsf{eval}}$, Eve is guaranteed to find such a query-answer pair.

The problem of revealing labels. The oracle defined as above can be used in particular as a random *injective* oracle, for example by using the $\mathcal{O}_{\mathsf{label}}$ suboracle. This is problematic because the BKSY proof strategy [17] cannot handle injective random oracles. The reason is that even in good rounds Eve might simulate an answer to the injective random oracle with the same response that was given to Bob in the real oracle for a different query. I.e., putting together the queries of the simulated Alice and the real Bob would result in a non-injective oracle and therefore perfect correctness of the key agreement scheme is insufficient to argue that the simulated Alice and the real Bob output the same shared key. This is not a concern for the $\mathcal{O}_{\mathsf{clabel}}/\mathcal{O}_{\mathsf{iconstrain}}$ oracle, since for these oracles also the inverse function can be queried.

Defining $\mathcal{O}_{\mathsf{label}}$ as a regular (not necessarily injective) random oracle does not resolve this issue. Consider the following simple candidate key agreement:

1. Bob samples a random key K and sends the label $L := \mathcal{O}_{\mathsf{label}}(K)$ to Alice.
2. Alice samples a random key K' and checks if she got the same label $L = \mathcal{O}_{\mathsf{label}}(K')$

(a) If that check succeeds, she sends Bob "`success`". Alice then outputs $\mathcal{O}_{\mathsf{eval}}(K', x)$ and Bob outputs $\mathcal{O}_{\mathsf{eval}}(K, x)$ for some fixed value x.
(b) If that check fails, Alice and Bob run a different key agreement protocol with perfect correctness.

This key agreement is perfectly correct, because even though Alice and Bob use different PRF keys in step Item 2a, they will get the same output, because both keys have the same label.

However, when in this protocol step Item 2a occurs, it would take Eve an exponential number of queries to learn the only intersection query $\mathcal{O}_{\mathsf{ieval}}(L, x)$. The reason is that the transcript contains only L, but neither of the keys K or K'. In each simulation, Eve will simulate Alice with a random key K'' and $\mathcal{O}'_{\mathsf{label}}(K'') = L$ (where $\mathcal{O}'_{\mathsf{label}}$ is the simulated $\mathcal{O}_{\mathsf{label}}$ oracle) and in each update query Eve will query $\mathcal{O}_{\mathsf{label}}(K'')$. But the probability that $\mathcal{O}_{\mathsf{label}}(K'') = L$. Each subsequent round will be almost the same, except that Eve will no longer consider K'' as candidate key that Alice sampled. Thus, it will take on average an exponential number of queries until Eve finds a key with the correct label and only then she can learn the shared key of Alice and Bob.

Of course, in the above example, the step Item 2a occurs only with negligible probability and, in fact, we do not believe that there actually exists a (perfectly correct) key agreement scheme relative to the oracle $\mathcal{O}$.[4] Nevertheless, this is a valid counterexample for the proof strategy outlined so far without an easy fix that works *in general*.

Our solution: Hiding the label. We resolve this issue by modifying the oracle to not reveal the labels. Therefore, we make the oracles $\mathcal{O}_{\mathsf{label}}$ and $\mathcal{O}_{\mathsf{clabel}}$ internal oracles (and therefore call them $\mathcal{O}_{\mathsf{ilabel}}$ and $\mathcal{O}_{\mathsf{iclabel}}$), that cannot be called directly, and instead give the adversary an oracle $\mathcal{O}_{\mathsf{same}}$ that inputs two (possibly constrained) keys and outputs 1 iff they belong to the same root PRF key, by making internal $\mathcal{O}_{\mathsf{ilabel}}$ and $\mathcal{O}_{\mathsf{iclabel}}$ queries.

With such an oracle, the above counterexample no longer works, because Alice and Bob have no means to be certain that they sampled the same PRF key (or one with the same label) without one party actually sending the PRF key.

Of course, Eve now will also be unable to learn labels and can only query $\mathcal{O}_{\mathsf{same}}$ to figure out if two (constrained) PRF keys belong to the same root. Thus Eve's simulated internal $\mathcal{O}_{\mathsf{ilabel}}$ oracle will answer with different labels as the real oracle, even on intersection queries. But Eve can learn the real *equality pattern* of labels in the update phase via the $\mathcal{O}_{\mathsf{same}}$ oracle and use this to make the equality pattern of her simulated $\mathcal{O}_{\mathsf{ilabel}}$ oracle consistent with the real oracle. This is sufficient for a faithful simulation of the oracles, because the equality pattern of the labels is now the only thing that matters for the public suboracles.

[4] It is known that an injective random oracle alone is insufficient for perfectly correct KA with the arguments in [48, 4].

3 Preliminaries

We use $\mathbb{N}_0$ for the set of natural numbers with zero and $\mathbb{N}_+$ for the set of natural numbers without zero. For a set S we use $\mathfrak{P}(S) := \{X \subseteq S\}$ to denote the power set of S. For $n, m \in \mathbb{Z}$ we define $[n, m] := \{x \in \mathbb{Z} \mid n \leq x \leq m\}$ and $[n] := [1, n]$. For a function $f : A \to B$ and $A' \subseteq A$ we use $f\big|_{A'}$ to denote the function $f\big|_{A'} : A' \to B$ with $f(x) = f\big|_{A'}(x)$ for all $x \in A'$.

Tuples. We write ε for the empty tuple. We use $(a, \ldots, b) \| (c, \ldots, d)$ to denote the tuple $(a, \ldots, b, c, \ldots, d)$. In particular $t \| \varepsilon = t$ for any tuple t. We say that a tuple t_1 is a prefix of t_2 if there exists a (possibly empty) tuple t_3 such that $t_2 = t_1 \| t_3$. For a set S and $n \in \mathbb{N}_0$, we use $S^{\leq n}$ to denote all tuples containing at most n elements, all contained in S. We use $(x_1, \ldots, x_m)_{|n}$ with $n \leq m$ to denote the n-element prefix $(x_1, \ldots, x_n)$ of a tuple and $\mathsf{Prefix}((x_1, \ldots, x_m)) := \{(x_1, \ldots, x_m)_{|n} \mid n \in [0, m]\}$. We write $(a, \ldots, b, \_, c, \ldots, d) \in S$ as shorthand for $\exists x.(a, \ldots, b, x, c, \ldots, d) \in S$.

3.1 Probability theory

Definition 1 (Statistical distance). *The statistical distance between two random variables X and Y is defined as*

$$\mathsf{SD}(X, Y) := \frac{1}{2} \sum_x |\Pr[X = x] - \Pr[Y = x]|.$$

We make use of the following basic properties for statistical distance.

Fact 2. For random variables X, X', Y, Z and a function f
(i) $\mathsf{SD}(f(X), f(X')) \leq \mathsf{SD}(X, X')$
(ii) $\mathsf{SD}((X, Y), (X', Y)) = \mathsf{SD}(X, X')$ if Y is independent of X and X'
(iii) $\mathsf{SD}(X, Z) \leq \mathsf{SD}(X, Y) + \mathsf{SD}(Y, Z)$ (triangle inequality)

3.2 Key agreement

Definition 2 (Key agreement). *A* key agreement *protocol for shared key space $\mathcal{K}$ consists of two interactive PPT algorithms (A, B) that both get 1^λ as input, then send messages to each other in $n := \mathsf{poly}(\lambda)$ rounds, and finally each output a candidate shared key. We denote these outputs with $k_A \in \mathcal{K}$ and $k_B \in \mathcal{K}$, respectively. We use $(k_A, k_B) \leftarrow \left[\mathsf{A}(1^\lambda) \leftrightarrows \mathsf{B}(1^\lambda)\right]$ to denote these keys for a random run of A and B. In a key agreement scheme relative to an oracle, A and B get access to an oracle $\mathcal{O}$.*

Definition 3 (Correctness). *A key agreement schemes is* ε-correct *iff $\Pr[k_A = k_B \neq \bot] \geq 1 - \varepsilon$, where the probability is taken over $(k_A, k_B) \leftarrow \left[\mathsf{A}(1^\lambda) \leftrightarrows \mathsf{B}(1^\lambda)\right]$ (and over the oracle $\mathcal{O}$, if the key agreement is relative to a randomized oracle). If $\varepsilon = 0$, we say that it is perfectly correct.*

Definition 4 (Security). *A key agreement scheme* $\mathsf{KA} = (\mathsf{A}, \mathsf{B})$ *is secure, if for every adversary* $\mathcal{A}$

$$\mathsf{Adv}^{\mathsf{eavesdrop}}_{\mathsf{KA},\mathcal{A}}(\lambda) := 2\left|\Pr[\mathcal{A}(1^\lambda, T, k_b) = b] - \frac{1}{2}\right| \leq \mathsf{negl}(\lambda)$$

for a negligible function negl*, where the randomness is taken over* $\mathcal{A}$*'s randomness,* $(k_A, k_B) \leftarrow \left[\mathsf{A}(1^\lambda) \leftrightarrows \mathsf{B}(1^\lambda)\right]$*,* $k_1 \xleftarrow{\$} \mathcal{K}$*, and* $b \xleftarrow{\$} \{0,1\}$*.* $\mathcal{A}$ *gets a transcript* T *of the messages exchanged in the run* $\mathsf{A}(1^\lambda) \leftrightarrows \mathsf{B}(1^\lambda)$ *and* $k_0 := k_B$ *or* $k_1 \xleftarrow{\$} \mathcal{K}$*. If the key agreement scheme is relative to an oracle,* $\mathcal{A}$ *get access to the oracle* $\mathcal{O}$ *but can make only polynomially many queries (and the randomness is also taken over the oracle* $\mathcal{O}$*, if* $\mathcal{O}$ *is randomized).*

The transcript T *is a list that contains an entry* `Alice`: x *for every message* x *sent by* A *and an entry* `Bob`: y *for every message* y *sent by* B *in* $\mathsf{A}(1^\lambda) \leftrightarrows \mathsf{B}(1^\lambda)$*, in the order in which these messages where sent.*

We use the wide-spread convention to call the user running A Alice, the user running B Bob, and the adversary Eve.

3.3 Constrained PRFs

We recall the definition of constrained pseudorandom functions (PRFs) from [14]. Syntactically, we represent constraint sets as subsets of the domain and constraint classes as sets of constraint sets.

Definition 5 (Constrained PRF). *A* constrained PRF *(CPRF) for domain* $\mathcal{X}_\lambda$*, co-domain* $\mathcal{Y}_\lambda$*, constraint class* $\mathcal{C}_\lambda \subseteq \mathfrak{P}(\mathcal{X}_\lambda)$*, a key space* $\mathcal{K}_\lambda$ *of a super-polynomial size, and constrained key space* $\mathcal{CK}_\lambda$ *of size at least* $|\mathcal{K}_\lambda| \cdot |\mathcal{C}_\lambda|$ *and with* $\mathcal{K}_\lambda \cap \mathcal{CK}_\lambda = \emptyset$ *consists of three PPT algorithms (or algorithms that make only polynomially many oracle queries)* $\mathsf{CPRF} = (\mathsf{KeyGen}, \mathsf{Constrain}, \mathsf{Eval})$ *where*

- $\mathsf{KeyGen}(1^\lambda)$ *takes the unary encoded security parameter* λ *and outputs a PRF (root) key* $K \in \mathcal{K}_\lambda$*. We assume that* K *implicitly contains* λ*.*
- $\mathsf{Constrain}(K, C \in \mathcal{C}_\lambda)$ *takes a PRF key* $K \in \mathcal{K}_\lambda$ *and a constraint* $C \in \mathcal{C}_\lambda$ *and outputs a constrained PRF key* $K\llbracket C \rrbracket \in \mathcal{CK}_\lambda$*. We assume that* $K\llbracket C \rrbracket$ *implicitly contains* λ*.*
- $\mathsf{Eval}(K, x \in \mathcal{X}_\lambda)$ *is deterministic and takes a (possibly constrained) PRF key* K *and* $x \in \mathcal{X}_\lambda$*, and outputs a value* $y \in \mathcal{Y}_\lambda$*.*

A CPRF relative to an oracle $\mathcal{O}$ *is defined identically, but all algorithms get oracle access to* $\mathcal{O}$*.*

A constrained PRF has to satisfy the following correctness property:

Definition 6 (Correctness). *A constrained PRF* CPRF *is* correct *iff there exists a negligible function* negl *such that for every* $C \in \mathcal{C}_\lambda$ *for*

- $K \leftarrow \mathsf{KeyGen}(1^\lambda)$*, and*

– $K\llbracket C\rrbracket \leftarrow \mathsf{Constrain}(K, C)$, *we have*

$$\Pr[\forall x \in C : \mathsf{Eval}(K, x) = \mathsf{Eval}(K\llbracket C\rrbracket, x)] \geq 1 - \mathsf{negl}(\lambda),$$

where the probability is taken over the randomness used to sample K *and* $K\llbracket C\rrbracket$ *(and over the randomness of the oracle* $\mathcal{O}$*, if the key agreement is relative to a randomized oracle). If* $\mathsf{negl} = 0$*, we call the scheme perfect correct.*

As security notion we consider in this work pseudorandomness against *unbounded* adversaries that can make at most polynomially (in λ) many queries to each of their oracles. The generic transformations in Appendices B to D can also be instantiated with schemes that only achieve security against PPT adversaries, but then, of course, the resulting scheme also achieves security only against PPT adversaries.

Definition 7 (Pseudorandomness). *A constrained PRF* CPRF *is* adaptively pseudorandom *iff for every adversary* $\mathcal{A}$

$$\mathsf{Adv}^{\mathsf{adpt\text{-}pr}}_{\mathsf{CPRF},\mathcal{A}}(\lambda) := \left|\Pr[\mathsf{Exp}^{\mathsf{adpt\text{-}pr}}_{\mathsf{CPRF},\mathcal{A}}(\lambda) \Rightarrow 1] - \frac{1}{2}\right| \leq \mathsf{negl}(\lambda),$$

for a negligible function negl*. The game* $\mathsf{Exp}^{\mathsf{adpt\text{-}pr}}_{\mathsf{CPRF},\mathcal{A}}(\lambda)$ *is defined in Figure 2. The probability is taken over the randomness of the adversary and the game (and the randomness of the oracle* $\mathcal{O}$*, if the key agreement is relative to a randomized oracle).*

Key delegation A CPRF supports key delegation if it is possible to generate a constrained key $K\llbracket C\rrbracket$ not only from the CPRF key K, but also from any constrained key $K\llbracket C'\rrbracket$ with $C \subseteq C'$.

Definition 8 (Delegatable CPRF). *A* delegatable CPRF *(in the ROM) for constraint class* $\mathcal{C}_\lambda$ *consist of three PPT algorithms* $\mathsf{DCPRF} = (\mathsf{KeyGen}, \mathsf{Constrain}, \mathsf{Eval})$ *such that*

– $(\mathsf{KeyGen}, \mathsf{Constrain}, \mathsf{Eval})$ *is a CPRF (see Definition 5) for constraint class* $\mathcal{C}_\lambda$ *and*
– $\mathsf{Constrain}(K\llbracket C\rrbracket, C')$ *can additionally be called with a constrained key* $K\llbracket C\rrbracket$ *for* $C \supseteq C'$ *with* $C, C' \in \mathcal{C}_\lambda$ *instead of the PRF key* K*. It outputs a constrained key* $K\llbracket C'\rrbracket$ *for* C'*.*

A delegatable CPRF relative to an oracle $\mathcal{O}$ *is defined identically, but all algorithms get oracle access to* $\mathcal{O}$*.*

Despite several previous works on delegatable CPRFs, only [28, 29] introduced a security model for delegatable CPRFs (although [23] seems to implicitly use the same model). Essentially, their model is identical to the selective pseudorandomness game, except that the adversary gets an additional key delegation oracle that

$\underline{\mathsf{Exp}^{\text{adpt-pr}}_{\mathsf{CPRF},\mathcal{A}}(\lambda)}$:
$f : \{0,1\}^* \dashrightarrow \{0,1\}^\lambda$
$K \leftarrow \mathsf{KeyGen}(1^\lambda)$
$b \xleftarrow{\$} \{0,1\}$
$R \leftarrow \emptyset$
$b' \leftarrow \mathcal{A}^{\mathcal{O}(\cdot),\textsc{Eval}(\cdot),\textsc{Constrain}(\cdot),\textsc{Chal}(\cdot)}(1^\lambda)$
if $x^\star \in R$ **then**
  return $b'' \xleftarrow{\$} \{0,1\}$
else
  return $b \stackrel{?}{=} b'$

$\underline{\textsc{Eval}(x)}$:
$R \leftarrow R \cup \{x\}$
return $\mathsf{Eval}(K, x)$

$\underline{\textsc{Constrain}(C)}$:
$R \leftarrow R \cup C$
return $\mathsf{Constrain}(K, C)$

$\underline{\textsc{Chal}(x^\star)}$: //only one query
if $b = 0$ **then**
  return $\mathsf{Eval}(K, x^\star)$
else
  return $y^\star \xleftarrow{\$} \mathcal{Y}_\lambda$

Fig. 2. The adaptive pseudorandomness experiment for constrained PRFs. The adversary can query CHAL once and all other oracles polynomially many times. If the CPRF is defined relative to an oracle $\mathcal{O}$, the adversary gets access to the oracle $\mathcal{O}$, too.

takes two constraints C, C' with $C \subseteq C'$. It then looks up if a constrained key has been generated for any C in a previous constrained key or key delegation query. If not, a constrained key for C is generated with the (unconstrained) CPRF key. The constrained key for C is then used to generate a constrained key for C' and this key is given to the adversary.

This model has the following significant limitations which make it unsuitable, for example for hierarchical ID-NIKE (see Appendix D).

1. It is impossible for the adversary to obtain a constrained key that has been generated by applying Constrain three or more times, without revealing an intermediate constrained key. This is because the key delegation oracle only allows the adversary to specify one intermediate constraint set.
2. For each constraint set C at most one key can be used for delegation. This is because the key delegation oracle always uses an already generated constrained key for C, if it exists, instead of giving the adversary the choice which previously generated constrained key should be used (or if a new constrained key should be generated) for delegation.

A more realistic security model, that is adequate for example for our application to HID-NIKE, needs to give the adversary the power to specify the entire "delegation path" of each constrained key and let the adversary decide adaptively which constrained keys along the delegation path should be revealed.

A similar problem arises when defining correctness for delegatable CPRFs. Again, [28, 29] is the only work that defines a formal correctness notion for delegatable CPRFs. However, their notion only requires correctness to hold for constrained keys that have been obtained by applying Constrain at most twice, which is not sufficient for HID-NIKE (with more than two levels).

Unfortunately, defining correctness and security notions for arbitrary delegation paths is quite cumbersome. Such a model has been defined for hierarchical identity-based encryption in [69]. However, we avoid this by using an approach that has become the standard for HIBE and also been introduced recently in the context of HID-NIKE in [22]. Namely, we ask for an additional property called *delegation invariance*, that essentially requires that delegated constrained keys are statistically close to keys freshly generated from the PRF key. This allows us to use the correctness and security model for non-delegatable CPRFs without modification, because an additional key delegation oracle would behave identical to the Constrain oracle (except with negligible probability). This definition is based on [22].

Definition 9 (Delegation invariance). *A delegatable CPRF* $\mathsf{DCPRF} = (\mathsf{KeyGen}, \mathsf{Constrain}, \mathsf{Eval})$ *for constraint class* $\mathcal{C}_\lambda$ *is* delegation invariant *iff*

- $\forall C, C' \in \mathcal{C}_\lambda$ *with* $C \supseteq C'$,
- $K \leftarrow \mathsf{KeyGen}(1^\lambda)$,
- $K[\![C']\!] \leftarrow \mathsf{Constrain}(K, C')$,
- $K[\![C]\!] \leftarrow \mathsf{Constrain}(K, C)$, *and*
- $K'[\![C']\!] \leftarrow \mathsf{Constrain}(K[\![C]\!], C')$, *we have*

$$(K, K[\![C]\!], K[\![C']\!]) \approx_s (K, K[\![C]\!], K'[\![C']\!]),$$

where the probability is taken over the randomness used to sample the variables K, $K[\![C']\!]$, $K[\![C]\!]$, *and* $K'[\![C']\!]$. *If the delegatable CPRF is defined relative to an oracle (e.g. a random oracle), we need*

$$(K, \mathcal{O}, K[\![C]\!], K[\![C']\!]) \approx_s (K, \mathcal{O}, K[\![C]\!], K'[\![C']\!]),$$

where $\mathcal{O}$ *is the entire function table of the oracle and all other variables are defined as above.*

We say that the delegatable CPRF is perfectly delegation invariant, iff $(K, [\mathcal{O},]K[\![C]\!], K[\![C']\!])$ *and* $(K, [\mathcal{O},]K[\![C]\!], K'[\![C']\!])$ *are identically distributed.*

Definition 10 (Correctness). *A* delegation invariant *delegatable CPRF* $\mathsf{DCPRF} = (\mathsf{KeyGen}, \mathsf{Constrain}, \mathsf{Eval})$ *is correct, if it is correct as a regular CPRF (see Definition 6).*

Definition 11 (Pseudorandomness). *A* delegation invariant *delegatable CPRF* $\mathsf{DCPRF} = (\mathsf{KeyGen}, \mathsf{Constrain}, \mathsf{Eval})$ *is selectively or adaptively pseudorandom, if it is selectively or adaptively pseudorandom, respectively, as a regular CPRF (see Definition 7).*

4 Impossibility of key agreement from CPRFs

In this section we introduce an oracle $\mathcal{O}$ such that constrained PRFs for any constraint class $\mathcal{C}_\lambda \subseteq \mathfrak{P}(\mathcal{X}_\lambda)$ exist relative to this oracle, even against unbounded adversaries that make at most polynomially many oracle queries. In this work we use the convention that adversaries are unbounded and make at most polynomially

many oracle queries. Our result also holds in the model where adversaries are PPT algorithms and get additionally a **PSPACE** oracle, when we additionally assume that each constraint $C \in \mathcal{C}_\lambda$ can be represented with polynomially many bits.

The oracle $\mathcal{O}$ is defined for an arbitrary super-polynomial CPRF key space $\mathcal{K}_\lambda$, input space $\mathcal{X}_\lambda$, output space $\mathcal{Y}_\lambda$ and constraint class $\mathcal{C}_\lambda$. It also needs a space of constrained keys $\mathcal{CK}_\lambda$ of size $|\mathcal{CK}_\lambda| \geq (|\mathcal{K}_\lambda| \cdot |\mathcal{C}_\lambda|)$ with $\mathcal{K}_\lambda \cap \mathcal{CK}_\lambda = \emptyset$, and label space $\mathcal{L}_\lambda$ of size $|\mathcal{L}_\lambda| = |\mathcal{K}_\lambda|$. Each CPRF key will have an associated label in our oracle. The purpose of this label is to serve as an identifier for CPRF root keys that allows us to determine if two constrained keys belong to the same CPRF root key without revealing this CPRF key.

The oracle $\mathcal{O}$ consists of the four public suboracles $\mathcal{O}_{\mathsf{constrain}}$, $\mathcal{O}_{\mathsf{eval}}$, $\mathcal{O}_{\mathsf{cset}}$ and $\mathcal{O}_{\mathsf{same}}$, that are defined using five internal suboracles $\mathcal{O}_{\mathsf{ieval}}$ and $\mathcal{O}_{\mathsf{iconstrain}}$, $\mathcal{O}_{\mathsf{ilabel}}$, $\mathcal{O}_{\mathsf{iclabel}}$, and $\mathcal{O}_{\mathsf{i[c]label}}$. That is, each $\mathcal{O}$ query specifies one of the four public suboracles to call and the inputs for this suboracle. The internal suboracles are not directly accessible and can only be called from other suboracles.

1. The internal suboracle $\mathcal{O}_{\mathsf{ieval}} : \mathcal{L}_\lambda \times \mathcal{X}_\lambda \to \mathcal{Y}_\lambda$ is a random function.
2. The internal suboracle $\mathcal{O}_{\mathsf{iconstrain}} : \mathcal{L}_\lambda \times \mathcal{C}_\lambda \to \mathcal{CK}_\lambda$ is a random injective function.
3. The internal suboracle $\mathcal{O}_{\mathsf{ilabel}} : \mathcal{K}_\lambda \to \mathcal{L}_\lambda$ is an arbitrary permutation.
4. The internal suboracle $\mathcal{O}_{\mathsf{iclabel}} : \mathcal{CK}_\lambda \to (\mathcal{L}_\lambda \times \mathcal{C}_\lambda) \cup \{\bot\}$ is the inverse of $\mathcal{O}_{\mathsf{iconstrain}}$ in the following sense: $\mathcal{O}_{\mathsf{iclabel}}(\overline{K}) = (L, C)$ iff $\mathcal{O}_{\mathsf{iconstrain}}(L, C) = \overline{K}$ and $\mathcal{O}_{\mathsf{iclabel}}(\overline{K}) = \bot$ iff $\overline{K}$ is not in the image of $\mathcal{O}_{\mathsf{iconstrain}}$.
5. The public suboracle $\mathcal{O}_{\mathsf{eval}} : (\mathcal{K}_\lambda \cup \mathcal{CK}_\lambda) \times \mathcal{X}_\lambda \to \mathcal{Y}_\lambda \cup \{\bot\}$ is implemented as follows:
   $\underline{\mathcal{O}_{\mathsf{eval}}(K, x) \text{ with } K \in \mathcal{K}_\lambda:}$
   $L \leftarrow \mathcal{O}_{\mathsf{ilabel}}(K)$
   return $\mathcal{O}_{\mathsf{ieval}}(L, x)$
   $\underline{\mathcal{O}_{\mathsf{eval}}(\overline{K}, x) \text{ with } \overline{K} \in \mathcal{CK}_\lambda}$
   $(L, C) \leftarrow \mathcal{O}_{\mathsf{iclabel}}(\overline{K})$
   if $x \in C$ **then return** $\mathcal{O}_{\mathsf{ieval}}(L, x)$ **else return** $\bot$
6. The public suboracle $\mathcal{O}_{\mathsf{constrain}} : (\mathcal{K}_\lambda \cup \mathcal{CK}_\lambda) \times \mathcal{C}_\lambda \to \mathcal{CK}_\lambda \cup \{\bot\}$ is implemented as follows:
   $\underline{\mathcal{O}_{\mathsf{constrain}}(K, C) \text{ with } K \in \mathcal{K}_\lambda:}$
   $L \leftarrow \mathcal{O}_{\mathsf{ilabel}}(K)$
   return $\mathcal{O}_{\mathsf{iconstrain}}(L, C)$
   $\underline{\mathcal{O}_{\mathsf{constrain}}(\overline{K}, C') \text{ with } \overline{K} \in \mathcal{CK}_\lambda:}$
   $(L, C) \leftarrow \mathcal{O}_{\mathsf{iclabel}}(\overline{K})$
   if $(L, C) = \bot$ **then return** $\bot$
   else if $C' \subseteq C$ **then return** $\mathcal{O}_{\mathsf{iconstrain}}(L, C)$ **else return** $\bot$
7. The public suboracle $\mathcal{O}_{\mathsf{cset}} : \mathcal{CK}_\lambda \to \mathcal{C}_\lambda$ is implemented as follows:
   $\underline{\mathcal{O}_{\mathsf{cset}}(\overline{K}):}$
   $(L, C) \leftarrow \mathcal{O}_{\mathsf{iclabel}}(\overline{K})$
   if $(L, C) \neq \bot$ **then return** C **else return** $\bot$

8. For convenience, we define the following internal suboracle $\mathcal{O}_{\mathsf{i[c]label}} : (\mathcal{K}_\lambda \cup \mathcal{CK}_\lambda) \to \mathcal{L}_\lambda \cup \{\bot\}$ as follows:
   $\underline{\mathcal{O}_{\mathsf{i[c]label}}(K) \text{ with } K \in \mathcal{K}_\lambda\text{:}}$
   return $\mathcal{O}_{\mathsf{ilabel}}(K)$
   $\underline{\mathcal{O}_{\mathsf{i[c]label}}(\overline{K}) \text{ with } \overline{K} \in \mathcal{CK}_\lambda\text{:}}$
   $(L, C) \leftarrow \mathcal{O}_{\mathsf{iclabel}}(\overline{K})$
   if $(L, C) \neq \bot$ **then return** L **else return** $\bot$
9. The public suboracle $\mathcal{O}_{\mathsf{same}} : (\mathcal{K}_\lambda \cup \mathcal{CK}_\lambda) \times (\mathcal{K}_\lambda \cup \mathcal{CK}_\lambda) \to \{0, 1\}$ is implemented as follows:
   $\underline{\mathcal{O}_{\mathsf{same}}(K_1, K_2)\text{:}}$
   return *1 iff* $\mathcal{O}_{\mathsf{i[c]label}}(K_1) = \mathcal{O}_{\mathsf{i[c]label}}(K_2) \neq \bot$

Note that only the first three oracles are randomized. The remaining three oracles are deterministic given the first three oracles. Also note that the permutation we pick for $\mathcal{O}_{\mathsf{ilabel}}$ does not influence the distribution of the public suboracles. The purpose of this oracle is to simplify bookkeeping in our impossibility result.

We call any oracle in the image of the above distribution of oracles a valid CPRF oracle.

Theorem 3. *Relative to the oracle $\mathcal{O}$ defined above, a perfectly correct, perfectly delegation invariant and adaptive pseudorandom delegatable CPRF for the domain $\mathcal{X}_\lambda$, co-domain $\mathcal{Y}_\lambda$, constraint class $\mathcal{C}_\lambda \subseteq \mathfrak{P}(\mathcal{X}_\lambda)$, key space $\mathcal{K}_\lambda$, and constrained key space $\mathcal{CK}_\lambda$ as used in $\mathcal{O}$ exists.*

Proof. The construction is trivial: $\mathsf{KeyGen}(1^\lambda)$ outputs a uniformly random key $K \xleftarrow{\$} \mathcal{K}_\lambda$, $\mathsf{Constrain}(K, C)$ returns $\mathcal{O}_{\mathsf{constrain}}(K, C)$, and $\mathsf{Eval}(K, x)$ returns $\mathcal{O}_{\mathsf{eval}}(K, x)$.

Clearly, this construction is perfectly correct. It is also perfectly delegation invariant, because there is just one valid constrained key for each PRF key and constraint set.

The construction is also adaptive pseudorandom. Let K be the CPRF key used in the adaptive pseudorandomness game. Consider an adversary $\mathcal{A}$ that makes at most q_e $\mathcal{O}_{\mathsf{eval}}$ and $\mathcal{O}_{\mathsf{constrain}}$ queries, and at most q_c CONSTRAIN and $\mathcal{O}_{\mathsf{constrain}}$ queries.

First, we bound the probability that the adversary makes an $\mathcal{O}_{\mathsf{same}}$ query where one of the two arguments is K. Until the first such query is made, the key K is uniformly random from the adversaries' perspective. Thus, the probability that an $\mathcal{O}_{\mathsf{same}}$ is the first query where one argument is K is at most $1/|\mathcal{K}_\lambda|$ and the probability that the adversary makes such a query is at most $q_s/|\mathcal{K}_\lambda|$, where q_s is the number of $\mathcal{O}_{\mathsf{same}}$ queries. This probability is negligible.

Next, we bound the probability that the adversary makes an $\mathcal{O}_{\mathsf{same}}$ query where one of the two arguments is a constrained key $\overline{K}$ where $\mathcal{O}_{\mathsf{same}}(K, \overline{K})$ and $x^\star \in \mathcal{O}_{\mathsf{cset}}(\overline{K})$. Until the first such query is made, each such constrained key is uniformly random in the set $\mathcal{CK}_\lambda \setminus \{\overline{K}_1, \ldots, \overline{K}_{q_c}\}$, where $\overline{K}_1, \ldots, \overline{K}_{q_c}$ are the constrained keys the adversary obtained via the CONSTRAIN or $\mathcal{O}_{\mathsf{constrain}}$ queries. Thus, the probability that an $\mathcal{O}_{\mathsf{same}}$ query is the first such query is at most

$|\mathcal{C}_\lambda|/(|\mathcal{CK}_\lambda| - q_c)$ and the probability that the adversary makes such a query is at most $q_s \cdot |\mathcal{C}_\lambda|/(|\mathcal{CK}_\lambda| - q_c)$, which is negligible.

If none of the above events occurs, the CPRF key K is still uniformly random from the adversaries perspective at the end of the game, and each constrained key for the root key K and a constraint set $C \ni x^\star$ is uniformly random in the set $\mathcal{CK}_\lambda \setminus \{\overline{K}_1, \ldots, \overline{K}_{q_c}\}$, where $\overline{K}_1, \ldots, \overline{K}_{q_c}$ are the constrained keys the adversary obtained via the CONSTRAIN or $\mathcal{O}_{\mathsf{constrain}}$ queries. Thus, the probability that the adversary makes an $\mathcal{O}_{\mathsf{eval}}(K, \cdot)$ or $\mathcal{O}_{\mathsf{constrain}}(K, \cdot)$ query is at most $q_e/|\mathcal{K}_\lambda|$ (which is negligible) and the probability that the adversary makes an $\mathcal{O}_{\mathsf{eval}}(\overline{K}, \cdot)$ or $\mathcal{O}_{\mathsf{constrain}}(\overline{K}, \cdot)$ query such that $\mathcal{O}_{\mathsf{constrain}}(K, C) = \overline{K}$ for some $C \in \mathcal{C}_\lambda$ and $x^\star \in C$ is at most $q_e/(|\mathcal{CK}_\lambda| - q_c)$ (which is also negligible).

When the adversary makes none of these queries, the adversary cannot cause an $\mathcal{O}_{\mathsf{ieval}}(\mathcal{O}_{\mathsf{ilabel}}(K), x^\star)$ query outside the challenge query $\textsc{Chal}(x^\star)$. Therefore, $\mathcal{O}_{\mathsf{ieval}}(\mathcal{O}_{\mathsf{ilabel}}(K), x^\star)$ is uniformly random in the adversary's perspective, which means the view of the adversary is independent of the challenge bit b of the adaptive pseudorandomness game and the adversary wins with probability 1/2. □

Before we proceed to prove that there is no information-theoretic key agreement relative to the oracle $\mathcal{O}$, we present some definitions and results about the oracle $\mathcal{O}$.

To describe query-answer pairs for the oracle $\mathcal{O}$, we use the sets $\mathsf{QA}_{\mathsf{ieval}}$, $\mathsf{QA}_{\mathsf{iconstrain}}$, $\mathsf{QA}_{\mathsf{ilabel}}$, and $\mathsf{QA}_{\mathsf{iclabel}}$ that store the query-answer pairs for the suboracles $\mathcal{O}_{\mathsf{ieval}}$, $\mathcal{O}_{\mathsf{iconstrain}}$, $\mathcal{O}_{\mathsf{ilabel}}$, and $\mathcal{O}_{\mathsf{iclabel}}$. This is sufficient, because the remaining two oracles are deterministic and implemented using calls to the oracles above (with polynomially many oracle queries). We use QA to denote the union of these four sets. The different syntax of the query-answer pairs allows us to derive $\mathsf{QA}_{\mathsf{ieval}}$, $\mathsf{QA}_{\mathsf{iconstrain}}$, $\mathsf{QA}_{\mathsf{ilabel}}$, and $\mathsf{QA}_{\mathsf{iclabel}}$ from QA when needed.

Definition 12. *Let $p : \mathcal{L}_\lambda \to \mathcal{L}_\lambda$ be a permutation on $\mathcal{L}_\lambda$ and QA a set of query-answer pairs for $\mathcal{O}$. Then define*

- $p(\mathsf{QA}_{\mathsf{ieval}}) := \{(p(L), x, y) \mid (L, x, y) \in \mathsf{QA}_{\mathsf{ieval}}\}$,
- $p(\mathsf{QA}_{\mathsf{iconstrain}}) := \{((p(L), C), K[\![C]\!]) \mid ((L, C), K[\![C]\!]) \in \mathsf{QA}_{\mathsf{iconstrain}}\}$,
- $p(\mathsf{QA}_{\mathsf{ilabel}}) := \{(p(K), L) \mid (K, L) \in \mathsf{QA}_{\mathsf{ilabel}}\}$, *and*
- $p(\mathsf{QA}_{\mathsf{iclabel}}) := \{(\overline{K}, (p(L), C)) \mid (\overline{K}, (L, C)) \in \mathsf{QA}_{\mathsf{iclabel}}\}$.

We write $p(\mathsf{QA}) := p(\mathsf{QA}_{\mathsf{ieval}}) \cup p(\mathsf{QA}_{\mathsf{iconstrain}}) \cup p(\mathsf{QA}_{\mathsf{ilabel}}) \cup p(\mathsf{QA}_{\mathsf{iclabel}})$ and $p(\mathcal{O})$ for the oracle that answers all queries according $p(\mathsf{QA}')$, where QA' is the entire function table of $\mathcal{O}$.

Clearly, if QA is a set of query-answer pairs of the oracle $\mathcal{O}$, then $p(\mathsf{QA})$ is a set of query-answer pairs of the $p(\mathcal{O})$ oracle.

Lemma 1. *Let $p : \mathcal{L}_\lambda \to \mathcal{L}_\lambda$ be a permutation on $\mathcal{L}_\lambda$. Then $\mathcal{O}$ and $p(\mathcal{O})$ answer all* public *suboracle queries identically.*

Proof. The oracles $\mathcal{O}_{\mathsf{eval}}$ and $\mathcal{O}_{\mathsf{constrain}}$ oracle derive a label via the $\mathcal{O}_{\mathsf{ilabel}}$ (or $\mathcal{O}_{\mathsf{iclabel}}$) query only to query the $\mathcal{O}_{\mathsf{ieval}}$ (or $\mathcal{O}_{\mathsf{iconstrain}}$) oracle with that respective

label. Permuting the labels consistently for $\mathcal{O}_{\mathsf{ilabel}}$ (and $\mathcal{O}_{\mathsf{iclabel}}$) and $\mathcal{O}_{\mathsf{ieval}}$ (and $\mathcal{O}_{\mathsf{iconstrain}}$) does not change their output.

The oracle $\mathcal{O}_{\mathsf{cset}}$ is clearly independent of the labels. The $\mathcal{O}_{\mathsf{same}}$ oracle compares the label of two (constrained) keys and applying a permutation to the labels does not change the result of this. □

This lemma justifies the following definition for consistency of query-answer pairs.

Definition 13. *We call two sets of query answer pairs* QA *and* QA' *consistent if there exists a permutation* $p : \mathcal{L}_\lambda \to \mathcal{L}_\lambda$ *and valid CPRF oracle* $\mathcal{O}$ *(an oracle with structure described above) such that* QA *and* $p(\mathsf{QA}')$ *are valid query-answer pairs for* QA*.*

To handle the redundancy between $\mathsf{QA}_{\mathsf{iconstrain}}$ and $\mathsf{QA}_{\mathsf{iclabel}}$, we define the closures $\widetilde{\mathsf{QA}}_{\mathsf{iconstrain}} := \mathsf{QA}_{\mathsf{iconstrain}} \cup \{((L, C), \overline{K}) \mid (\overline{K}, (L, C)) \in \mathsf{QA}_{\mathsf{iclabel}}\}$ and $\widetilde{\mathsf{QA}}_{\mathsf{iclabel}} := \mathsf{QA}_{\mathsf{iclabel}} \cup \{(\overline{K}, (L, C)) \mid ((L, C), \overline{K}) \in \mathsf{QA}_{\mathsf{iconstrain}}\}$ (allowing $(L, C) = \bot$ in both definitions). We write $\widetilde{\mathsf{QA}}$ to denote the union of $\mathsf{QA}_{\mathsf{ieval}}$, $\widetilde{\mathsf{QA}}_{\mathsf{iconstrain}}$, $\mathsf{QA}_{\mathsf{ilabel}}$, and $\widetilde{\mathsf{QA}}_{\mathsf{iclabel}}$. We say that the inverse query for $\mathcal{O}_{\mathsf{iclabel}}(\overline{K}) = (L, C)$ is $\mathcal{O}_{\mathsf{iconstrain}}(L, C) = \overline{K}$ and vice versa.

For convenience, we define $\mathsf{KeyLabelPairs}(\mathsf{QA}) := \mathsf{QA}_{\mathsf{ilabel}} \cup \{(\overline{K}, L) \mid \exists C \in \mathcal{C}_\lambda : (\overline{K}, (L, C) \in \widetilde{\mathsf{QA}}_{\mathsf{iclabel}}\}$ and $\mathsf{Labels}(\mathsf{QA}) := \{L \mid \exists K \in (\mathcal{K}_\lambda \cup \mathcal{CK}_\lambda) : (K, \overline{K}) \in \mathsf{KeyLabelPairs}(\mathsf{QA})\}$.

Definition 14. *We say that a permutation* $p : \mathcal{L}_\lambda \to \mathcal{L}_\lambda$ *is* label consistent *for* QA *and* QA' *iff* $\mathsf{KeyLabelPairs}(\mathsf{QA})$ *and* $\mathsf{KeyLabelPairs}(p(\mathsf{QA}'))$ *is consistent, i.e. for all* $(K, L) \in \mathsf{KeyLabelPairs}(\mathsf{QA}), (K, L') \in \mathsf{KeyLabelPairs}(\mathsf{QA}')$ *we have* $p(L') = L$*.*

Lemma 2. *Let* QA *and* QA' *be valid sets of query-answer pairs for oracles* $\mathcal{O}$ *and* $\mathcal{O}'$ *of the above structure. When* QA *and* QA' *are inconsistent, at least one of the following events has to occur:*

1. *There is no label consistent permutation for* QA *and* QA' *or*
2. *There exist* $((L, x), y) \in \mathsf{QA}_{\mathsf{ieval}}$ *and* $((L', x), y') \in \mathsf{QA}'_{\mathsf{ieval}}$ *with* $y \neq y'$ *and for every label consistent permutation* p *we have* $p(L) = L'$*.*
3. *There exist* $((L, C), \overline{K}) \in \widetilde{\mathsf{QA}}_{\mathsf{iconstrain}}$ *and* $((L', C), \overline{K}') \in \mathsf{QA}'_{\mathsf{iconstrain}}$ *with* $\overline{K} \neq \overline{K}'$ *and for every label consistent permutation* p *we have* $p(L) = L'$*.*
4. *There exist* $(\overline{K}, (\_, C)) \in \widetilde{\mathsf{QA}}_{\mathsf{iclabel}}$ *and* $(\overline{K}, (\_, C')) \in \mathsf{QA}'_{\mathsf{iclabel}}$ *with* $C \neq C'$*.*

Proof. We show the contraposition. Assume p is a label consistent permutation for QA and QA', and none of the events 2—4 occur. We need to show that there exists an oracle $\mathcal{O}$ such that both QA and $p(\mathsf{QA}')$ are valid sets of query-answer pairs for this oracle. It suffices to show this for the $\mathcal{O}_{\mathsf{ieval}}$, $\mathcal{O}_{\mathsf{iconstrain}}$, $\mathcal{O}_{\mathsf{ilabel}}$ suboracles, since all other suboracles of $\mathcal{O}$ are defined based on these suboracles.

We define $\mathcal{O}_{\mathsf{ieval}}(L, x) := y$ iff $(L, x, y) \in \mathsf{QA}_{\mathsf{ieval}}$ or $(p(L), x, y') \in \mathsf{QA}'_{\mathsf{ieval}}$. This is well-defined when event 2 does not occur. On all other inputs $\mathcal{O}_{\mathsf{ieval}}$ can be defined arbitrarily.

We define $\mathcal{O}_{\mathsf{iconstrain}}(L, C) := \overline{K}$ iff $(L, C, \overline{K}) \in \widetilde{\mathsf{QA}}_{\mathsf{iconstrain}}$ or $(p(L), C, \overline{K}') \in \widetilde{\mathsf{QA}}'_{\mathsf{iconstrain}}$. This is well-defined when event 3 does not occur. Moreover, this partial function is injective, because for $(L_1, C_1, \overline{K}) \in \widetilde{\mathsf{QA}}_{\mathsf{iconstrain}}$ and $(p(L_2), C_2, \overline{K}) \in \widetilde{\mathsf{QA}}'_{\mathsf{iconstrain}}$ we have $L_1 = p(L_2)$ since p is label consistent permutation and $C_1 = C_2$ when event 4 does not occur. $\mathcal{O}_{\mathsf{iconstrain}}$ can be extended arbitrarily to a permutation.

We define $\mathcal{O}_{\mathsf{ilabel}}(K) = L$ iff $(K, L) \in \mathsf{QA}_{\mathsf{ilabel}}$ or $(K, p(L)) \in \mathsf{QA}'_{\mathsf{ilabel}}$. This is well-defined since p is a label consistent permutation. □

Theorem 4. *Relative to the oracle $\mathcal{O}$ defined above, no perfectly correct key agreement scheme exists. More precisely, for each perfectly correct key agreement scheme in the $\mathcal{O}$ oracle model there exist an adversary (Eve) which makes $\mathcal{O}(q_A^2 q_B^2)$ queries (where q_A and q_B are the number of $\mathcal{O}$ oracle queries of Alice and Bob make, respectively) and breaks the security of the key agreement scheme with probability* 1*.*

Proof. Let (A, B) be a key agreement scheme relative to $\mathcal{O}$. We describe an unbounded adversary Eve that can break this scheme with polynomially many $\mathcal{O}$ queries.

Eve proceeds in $2q_B + 1$ rounds. Eve keeps track of all $\mathcal{O}$ query-answer pairs she has learned in each round in sets QA_i ($i \in [2q_B + 1]$) and all candidate shared keys in a multiset ShK. Initially, $\mathsf{QA}_1 := \mathsf{ShK} := \emptyset$. For all $i \in [2q_B + 1]$ Eve executes the following two phases.

1. In the simulation phase, Eve samples a random run of Alice (that is, random coins for Alice and answers for the $\mathcal{O}$ queries that she makes when running A), conditioned on being consistent with the transcript T that is given to Eve and the queries QA that Eve has learned so far.
   In more detail, Eve samples a uniformly random tape and an oracle $\mathcal{O}'_i$ such that $\mathsf{QA}_i \subseteq \mathcal{O}'_i$. Eve then runs $\mathsf{A}^{\mathcal{O}'_i}$. Whenever A sends a message to B, Eve checks if this message is identical to the message that Alice has sent in the real transcript T at this point. If this in not the case, Eve restarts the simulation phase. To guarantee termination, we make sure that Eve does not try the exact same combination of random tape and oracle $\mathcal{O}'_i$ again in any subsequent run of the simulation phase. Whenever A is expecting a message from B, Eve feeds to A the message that Bob has sent in the real transcript T at this point. Whenever A makes an oracle query, Eve answers this with $\mathcal{O}'_i$. Let QA'_i be the set of query-answer pairs made by A in the final simulation. Finally, when Alice outputs a key k_A, Eve stores k_A in ShK. Note that Eve makes no queries to the real oracle $\mathcal{O}$ in this phase.
2. In the update phase, Eve prepares QA_{i+1} for the next round by setting $\mathsf{QA}_{i+1} := \mathsf{QA}_i$ and then making the following queries to the real oracle $\mathcal{O}$ to update QA_{i+1}. In the last round, this update phase is omitted.
   (a) **for** $(K, \_) \in \mathsf{QA}'_{\mathsf{ilabel},i} \setminus \mathsf{QA}_{\mathsf{ilabel},i}$ **do**
   if $\exists (K', L) \in \mathsf{KeyLabelPairs}(\mathsf{QA}) : \mathcal{O}_{\mathsf{same}}(K, K') = 1$ **then**
   $\mathsf{QA}_{\mathsf{ilabel},i+1} \leftarrow \mathsf{QA}_{\mathsf{ilabel},i+1} \cup \{(K, L)\}$
   else pick $L \in \mathcal{L}_\lambda \setminus \mathsf{Labels}(\mathsf{QA})$

$\mathsf{QA}_{\mathsf{ilabel},i+1} \leftarrow \mathsf{QA}_{\mathsf{ilabel},i+1} \cup \{(K, L)\}$

(b) **for** $(\overline{K}, (\_, \_,)) \in \mathsf{QA}'_{\mathsf{iclabel},i} \setminus \mathsf{QA}_{\mathsf{iclabel},i}$ **do**
  $C := \mathcal{O}_{\mathsf{cset}}(\overline{K})$
  if $C = \bot$ **then** $\mathsf{QA}_{\mathsf{iclabel},i+1} \leftarrow \mathsf{QA}_{\mathsf{iclabel},i+1} \cup \{(\overline{K}, \bot)\}$
  else if $\exists (K', L) \in \mathsf{KeyLabelPairs}(\mathsf{QA}) : \mathcal{O}_{\mathsf{same}}(\overline{K}, K') = 1$ **then**
    $\mathsf{QA}_{\mathsf{iclabel},i+1} \leftarrow \mathsf{QA}_{\mathsf{iclabel},i+1} \cup \{(\overline{K}, (L, C)\}$
  else pick $L \in \mathcal{L}_\lambda \setminus \mathsf{Labels}(\mathsf{QA})$
    $\mathsf{QA}_{\mathsf{iclabel},i+1} \leftarrow \mathsf{QA}_{\mathsf{iclabel},i+1} \cup \{(\overline{K}, (L, C)\}$

(c) **for** $((L, C'), \_) \in \mathsf{QA}'_{\mathsf{iconstrain},i} \setminus \mathsf{QA}_{\mathsf{iconstrain},i}$ **do**
  i. **if** $\exists K \in \mathcal{K}_\lambda : (K, L) \in \mathsf{QA}_{\mathsf{ilabel}}$ **then**
    $\overline{K} \leftarrow \mathcal{O}_{\mathsf{constrain}}(K, C')$
    $\mathsf{QA}_{\mathsf{iconstrain},i+1} \leftarrow \mathsf{QA}_{\mathsf{iconstrain},i+1} \cup \{((L, C'), \overline{K})\}$
  ii. **else if** $\exists \overline{K} \in \mathcal{CK}_\lambda : ((L, C), \overline{K}) \in \mathsf{QA}_{\mathsf{ilabel}} \wedge C' \subseteq C$ **then**
    $\overline{K}' \leftarrow \mathcal{O}_{\mathsf{constrain}}(\overline{K}, C')$
    $\mathsf{QA}_{\mathsf{iconstrain},i+1} \leftarrow \mathsf{QA}_{\mathsf{iconstrain},i+1} \cup \{((L, C'), \overline{K}')\}$

(d) **for** $((L, x), \_) \in \mathsf{QA}'_{\mathsf{ieval},i} \setminus \mathsf{QA}_{\mathsf{ieval},i}$ **do**
  i. **if** $\exists K \in \mathcal{K}_\lambda : (K, L) \in \mathsf{QA}_{\mathsf{ilabel}}$ **then**
    $y \leftarrow \mathcal{O}_{\mathsf{eval}}(K, x)$
    $\mathsf{QA}_{\mathsf{ieval},i+1} \leftarrow \mathsf{QA}_{\mathsf{ieval},i+1} \cup \{((K, x), y)\}$
  ii. **else if** $\exists \overline{K} \in \mathcal{CK}_\lambda : ((L, C), \overline{K}) \in \mathsf{QA}_{\mathsf{ilabel}} \wedge x \in C$ **then**
    $y \leftarrow \mathcal{O}_{\mathsf{eval}}(K, x)$
    $\mathsf{QA}_{\mathsf{ieval},i+1} \leftarrow \mathsf{QA}_{\mathsf{ieval},i+1} \cup \{((K, x), y)\}$

Finally, Eve outputs the majority key in ShK.

In the following, when we refer to QA_i, QA'_i or ShK, we refer to the content of these variables at the end of Eve's execution.

Claim 5. Eve always terminates and makes at most $\mathcal{O}(q_A^2 q_B^2)$ $\mathcal{O}$ oracle queries.

Proof. Eve cannot keep restarting the simulation phase forever, because there is always the possibility that Eve picks the exact same random tape for Alice and oracle $\mathcal{O}'$ as in the real run of the key agreement scheme. In this case, the simulation will not have to be restarted. Since we avoid trying a combination of random tapes and oracle queries that lead to a restart more than once, each simulation phase will eventually terminate.

In the i-th round, Eve adds at most q_A query-answer pairs to QA_{i+1}, thus $|\mathsf{QA}_{i+1}| \leq iq_A$.

Eve queries $\mathcal{O}$ only in the update phase and makes in the i-th round at most $q_A(|\mathsf{KeyLabelPairs}(\mathsf{QA}_{i+1})| + 1) \leq q_A((i+1)q_A + 1)$ queries. Thus, the total number of $\mathcal{O}$ queries is bounded by $2q_B q_A((2q_B + 1)q_A + 1) \in \mathcal{O}(q_A^2 q_B^2)$. □

Claim 6. For each $i \in [2q_B + 1]$ there is a label-consistent permutation for QA_i and the real random oracle $\mathcal{O}$.

Proof. For $i = 1$ the statement is trivially true, since $\mathsf{QA}_1 = \emptyset$. We show that whenever Eve adds a query-answer pair to QA_i for any $i \in [2q_B + 1]$ the statement is preserved.

When Eve adds (K, L) to $\mathsf{QA}_{\mathsf{ilabel},i}$ (step 2a) and there exists $(K', L') \in \mathsf{KeyLabelPairs}(\mathsf{QA}_i)$ such that $\mathcal{O}_{\mathsf{i[c]label}}(K) = \mathcal{O}_{\mathsf{i[c]label}}(K')$, then $\mathcal{O}_{\mathsf{same}}(K, K') = 1$ and thus Eve picks $L = L'$ and any permutation p that was label consistent before (K, L) was added to QA_i will also be label consistent after (K, L) was added to QA_i.

On the other hand, when for all $(K', L') \in \mathsf{KeyLabelPairs}(\mathsf{QA}_i)$ we have $\mathcal{O}_{\mathsf{i[c]label}}(K) \neq \mathcal{O}_{\mathsf{i[c]label}}(K')$, then $\mathcal{O}_{\mathsf{same}}(K, K') = 0$ and Eve picks a fresh label $L \in \mathcal{L}_\lambda \setminus \mathsf{Labels}(\mathsf{QA}_i)$. Let p be a permutation that was label consistent before Eve added (K, L) to $\mathsf{QA}_{\mathsf{ilabel},i}$ and let $L_{\mathsf{real}} := \mathcal{O}_{\mathsf{ilabel}}(K)$. Let p' be the permutation that is identically to p, except that it swaps the $p(L)$ and L_{real} outputs. Formally, $p'(L) := L_{\mathsf{real}}$, $p'(p^{-1}(L_{\mathsf{real}})) := p(L)$ and $p'(L') = p(L')$ for all $L' \in \mathcal{L}_\lambda \setminus \{L, p^{-1}(L_{\mathsf{real}})\}$. Since $L \notin \mathsf{Labels}(\mathsf{QA}_i)$, p' is also label consistent for QA_i before Eve added (K, L) to $\mathsf{QA}_{\mathsf{ilabel},i}$. Since $p'(L) := \mathcal{O}_{\mathsf{ilabel}}(K)$, it is still label consist after Eve has added (K, L) to $\mathsf{QA}_{\mathsf{ilabel},i}$.

The argument for query-answer pairs of $\mathsf{QA}_{\mathsf{iclabel},i}$ (step 2b) is analogous to the above argument for query-answer pairs of $\mathsf{QA}_{\mathsf{ilabel},i}$.

When Eve adds a query-answer pair $((L, C'), K)$ to $\mathsf{QA}_{\mathsf{iconstrain},i}$ (step 2c), there has to be a (possibly constrained) key K with $(K, L) \in \mathsf{KeyLabelPairs}(\mathsf{QA}_i)$ before Eve adds $((L, C'), K)$ to $\mathsf{QA}_{\mathsf{iconstrain},i}$. Thus, such a query-answer pair will not add a new key–label pair.

Finally, the query-answer pairs in $\mathsf{QA}_{\mathsf{ieval},i}$ are irrelevant for the key-label pairs of QA_i. □

Let p_i be from now on a fixed label-consistent permutation for QA_i and $\mathcal{O}$ (which is guaranteed to exist by Claim 6).

Claim 7. For each $i \in [2q_B + 1]$ $\mathcal{O}$ and QA_i are consistent.

Proof. We show consistency under the label permutation p_i for each query-answer pair Eve adds to QA_i with a case distinction by the type of query.

Case 1: (K, L) **is added to** $\mathsf{QA}_{\mathsf{ilabel},i}$ **(step 2a).** This preserves consistency since p_i is label consistent.

Case 2: $(\overline{K}, (L, C))$ **is added to** $\mathsf{QA}_{\mathsf{iclabel},i}$ **(step 2b).** Since $C = \mathcal{O}_{\mathsf{cset}}(\overline{K})$ and p_i is label consistent, consistency for this query-answer pair follows.

Case 3: $((L, C), \overline{K})$ **is added to** $\mathsf{QA}_{\mathsf{iconstrain},i}$ **(step 2c).** Since $\overline{K} = \mathcal{O}_{\mathsf{constrain}}(K, C)$ with $\mathcal{O}_{\mathsf{i[c]label}}(K) = p_i(L)$ and $C \subseteq \mathcal{O}_{\mathsf{cset}}(K)$ if K is a constrained key, consistency for this query-answer pair follows.

Case 4: $((L, x), y)$ **is added to** $\mathsf{QA}_{\mathsf{ieval},i}$ **(step 2d).** Since $y = \mathcal{O}_{\mathsf{eval}}(K, C)$ with $\mathcal{O}_{\mathsf{i[c]label}}(K) = p_i(L)$ and $x \in \mathcal{O}_{\mathsf{cset}}(K)$ if K is a constrained key, consistency for this query-answer pair follows.

□

Let QA_B be the set of query-answer of the real execution of Bob. We say that round i is *good*, if at the end of the simulation phase we have no intersection queries between $p_i(\mathsf{QA}' \setminus \widetilde{\mathsf{QA}})$ and $\widetilde{\mathsf{QA}}_B$.

Claim 8. At least $q_B + 1$ out of the $2q_B + 1$ rounds are good.

Proof. The set QA_B of query-answer pairs of Bob stays fixed for all rounds. If the i-th round is bad, there is at least one query $\widehat{x}$ with $(\widehat{x}, \_) \in p_i(\widetilde{\mathsf{QA}}_i' \setminus \widetilde{\mathsf{QA}}_{i-1})$ and $(\widehat{x}, \_) \in \widetilde{\mathsf{QA}}_\mathsf{B}$. We say that the first such query in the simulated run of Alice is responsible for the bad round. We show in the following that Eve will query $\mathcal{O}$ on $\widehat{x}$ in the update phase and add the query-answer pair to QA_i. Thus, $\widehat{x}$ cannot be responsible for any further bad round. Moreover, if $\widehat{x}$ is an $\mathcal{O}_{\mathsf{iconstrain}}$ or $\mathcal{O}_{\mathsf{iclabel}}$ query, the inverse query also cannot cause a bad round, because the inverse query also becomes part of $\widetilde{\mathsf{QA}}$. Consequently, each of the at most q_B queries in QA_B can cause at most one bad round.

Let $\widehat{x}$ be the first query in the simulated run of Alice with $(\widehat{x}, \_) \in p_i(\mathsf{QA}_i' \setminus \widetilde{\mathsf{QA}}_{i-1}) \cap \widetilde{\mathsf{QA}}_\mathsf{B}$. We show that Alice adds $(\widehat{x}, \_)$ in the update phase to QA_i by a case distinction over the type of the query.

Case 1: $\widehat{x}$ is a $\mathcal{O}_{\mathsf{ilabel}}(K)$ query. In step Item 2a Eve adds (K, L) for some label L to $\mathsf{QA}_{\mathsf{ilabel},i}$.

Case 2: $\widehat{x}$ is a $\mathcal{O}_{\mathsf{iclabel}}(\overline{K})$ query. In step Item 2b Eve adds $(\overline{K}, \bot)$ or $(\overline{K}, (L, C))$ for some $L \in \mathcal{L}_\lambda$ and $C \in \mathcal{C}_\lambda$ to $\mathsf{QA}_{\mathsf{iclabel},i}$.

Case 3: $\widehat{x}$ is a $\mathcal{O}_{\mathsf{iconstrain}}(L, C')$ query. Such a query cannot be made directly by Alice. Instead, Alice can cause this query only indirectly with a $\mathcal{O}_{\mathsf{constrain}}(K, C')$ query. The $\mathcal{O}_{\mathsf{constrain}}$ query makes before answering the $\mathcal{O}_{\mathsf{iconstrain}}(L, C')$ query either a $L \leftarrow \mathcal{O}_{\mathsf{ilabel}}(K)$ query or a $(L, C) \leftarrow \mathcal{O}_{\mathsf{iclabel}}(K)$ query with $C' \subseteq C$ (depending on whether K is a constrained key or not).

If $(K, p_i(L)) \in \mathsf{KeyLabelPairs}(\widetilde{\mathsf{QA}}_\mathsf{B})$, then this $\mathcal{O}_{\mathsf{ilabel}}(K)$ or $\mathcal{O}_{\mathsf{iclabel}}(K)$ query must therefore be an earlier intersection query. Since we assume that $\mathcal{O}_{\mathsf{iclabel}}(\overline{K})$ is the first $\mathcal{O}_{\mathsf{iconstrain}}(L, C')$ intersection query not contained in QA_{i-1}, we can conclude that $(K, L) \in \mathsf{QA}_{\mathsf{ilabel},i-1}$ or $(K, (L, C)) \in \mathsf{QA}_{\mathsf{iclabel},i-1}$ with $C' \subseteq C$. Thus, one of the cases 2(c)i or 2(c)ii occurs and Eve finds a (possibly constrained) key K with label L that allows her to evaluate $\overline{K} \leftarrow \mathcal{O}_{\mathsf{constrain}}(K, C')$ and store $((L, C'), \overline{K})$ in $\mathsf{QA}_{\mathsf{iconstrain}}$.

On the other hand, if $(K, p_i(L)) \notin \mathsf{KeyLabelPairs}(\widetilde{\mathsf{QA}}_\mathsf{B})$, since p_i is label-consistent for QA and QA_B, any $\mathcal{O}_{\mathsf{constrain}}(K)$ query made by Bob will result in a $\mathcal{O}_{\mathsf{iconstrain}}(L', C')$ with $L' \neq p_i(L)$. Therefore, $\mathcal{O}_{\mathsf{iconstrain}}(L, C')$ cannot be an intersection query.

Case 4: $\widehat{x}$ is a $\mathcal{O}_{\mathsf{ieval}}(L, x)$ query. The argument is analogous to the previous case.

□

Claim 9. In each good round i, QA_i' and QA_B are consistent (in the sense of Definition 13).

Proof. First note that in a good round we have no intersection queries between $p_i(\mathsf{QA}' \setminus \widetilde{\mathsf{QA}})$ and $\widetilde{\mathsf{QA}}_B$ and therefore also no intersection queries between $p_i(\widetilde{\mathsf{QA}}' \setminus \widetilde{\mathsf{QA}})$ and $\widetilde{\mathsf{QA}}_B$, because for any query in $\widetilde{\mathsf{QA}}_B \setminus p_i(\widetilde{\mathsf{QA}})$ the inverse query is also in $\widetilde{\mathsf{QA}}_B \setminus p_i(\widetilde{\mathsf{QA}})$.

We use Lemma 2 and show that none of the cases that can cause an inconsistency to occur.

- The permutation p_i is label-consistent for QA_i and QA_B by Claim 6. Since the i-th round is good, we have $(K, L) \in \mathsf{KeyLabelPairs}(\mathsf{QA}'_i) \wedge (K, L') \in \mathsf{KeyLabelPairs}(\mathsf{QA}_\mathsf{B})$ whenever we have $(K, L) \in \mathsf{KeyLabelPairs}(\mathsf{QA}_i) \wedge (K, L') \in \mathsf{KeyLabelPairs}(\mathsf{QA}_\mathsf{B})$. Therefore, p_i is also label-consistent for QA'_i and QA_B.
- If $((L, x), y) \in \mathsf{QA}'_{\mathsf{ieval},i}$ and $((p_i(L), x), y') \in \mathsf{QA}_{\mathsf{ieval},\mathsf{B}}$ then $((L, x), y) \in \mathsf{QA}_{\mathsf{ieval},i}$ by definition of a good round and therefore $y = y'$ by Claim 7.
- Similarly, if $((L, C), \overline{K}) \in \widetilde{\mathsf{QA}}'_{\mathsf{iconstrain},i}$ and $((p_i(L), C), \overline{K}') \in \widetilde{\mathsf{QA}}_{\mathsf{iconstrain},\mathsf{B}}$ then $((L, C), \overline{K}) \in \widetilde{\mathsf{QA}}_{\mathsf{iconstrain},i}$ by definition of a good round, and therefore $\overline{K} = \overline{K}'$ by Claim 7.
- Finally, if $(\overline{K}, (\_, C)) \in \widetilde{\mathsf{QA}}'_{\mathsf{iclabel},i}$ and $(\overline{K}, (\_, C')) \in \widetilde{\mathsf{QA}}_{\mathsf{iclabel},\mathsf{B}}$, then $(\overline{K}, (\_, C)) \in \widetilde{\mathsf{QA}}_{\mathsf{iclabel},i}$ by definition of a good round and therefore $C = C'$ by Claim 7.

□

Claim 9 shows, together with Lemma 1, that in each good round i there exists a valid CPRF oracle $\mathcal{O}_i$ that answers all queries made by the real Bob identically to $\mathcal{O}$ and all queries made by the simulated Alice as in the simulation of round i. Since KA is perfectly correct, this shows that in all good rounds the simulated Alice will output the same shared key as Bob in the real execution. Therefore, this will be the majority value in ShK by Claim 8 and Eve will output it. □

Combining Theorems 3 and 4 yields the following corollary.

Corollary 1. *There is no black-box construction of a secure and perfectly correct key agreement protocol from delegatable CPRFs for any constraint class.*

Acknowledgments. I would like to thank Mohammad Hajiabadi and Dennis Hofheinz for discussion about topics covered in this work and Noam Mazor for pointing out a flaw in an earlier version of this work. The author was supported by SNSF Postdoc.Mobility grant no. 235425. Parts of this work have appeared in the author's PhD thesis [52].

References

[1] Hamza Abusalah, Georg Fuchsbauer, and Krzysztof Pietrzak. "Constrained PRFs for Unbounded Inputs". In: *CT-RSA 2016*. Ed. by Kazue Sako. Vol. 9610. LNCS. Springer, Cham, 2016, pp. 413–428. DOI: 10.1007/978-3-319-29485-8_24.

[2] Nuttapong Attrapadung, Takahiro Matsuda, Ryo Nishimaki, Shota Yamada, and Takashi Yamakawa. "Constrained PRFs for NC^1 in Traditional Groups". In: *CRYPTO 2018, Part II*. Ed. by Hovav Shacham and Alexandra Boldyreva. Vol. 10992. LNCS. Springer, Cham, Aug. 2018, pp. 543–574. DOI: 10.1007/978-3-319-96881-0_19.

[3] Abhishek Banerjee, Georg Fuchsbauer, Chris Peikert, Krzysztof Pietrzak, and Sophie Stevens. "Key-Homomorphic Constrained Pseudorandom Functions". In: *TCC 2015, Part II*. Ed. by Yevgeniy Dodis and Jesper Buus Nielsen. Vol. 9015. LNCS. Springer, Berlin, Heidelberg, Mar. 2015, pp. 31–60. DOI: 10.1007/978-3-662-46497-7_2.

[4] Boaz Barak and Mohammad Mahmoody-Ghidary. "Merkle Puzzles Are Optimal - An $O(n^2)$-Query Attack on Any Key Exchange from a Random Oracle". In: *CRYPTO 2009*. Ed. by Shai Halevi. Vol. 5677. LNCS. Springer, Berlin, Heidelberg, Aug. 2009, pp. 374–390. DOI: 10.1007/978-3-642-03356-8_22.

[5] Mihir Bellare and Chanathip Namprempre. "Authenticated Encryption: Relations among notions and analysis of the generic composition paradigm". In: *ASIACRYPT 2000*. Ed. by Tatsuaki Okamoto. Vol. 1976. LNCS. Springer, Berlin, Heidelberg, Dec. 2000, pp. 531–545. DOI: 10.1007/3-540-44448-3_41.

[6] Mengda Bi, Chenxin Dai, and Yaohua Ma. *Limits on the Power of Private Constrained PRFs*. Cryptology ePrint Archive, Report 2025/1196. 2025. URL: https://eprint.iacr.org/2025/1196.

[7] Rolf Blom. "An Optimal Class of Symmetric Key Generation Systems". In: *EUROCRYPT'84*. Ed. by Thomas Beth, Norbert Cot, and Ingemar Ingemarsson. Vol. 209. LNCS. Springer, Berlin, Heidelberg, Apr. 1985, pp. 335–338. DOI: 10.1007/3-540-39757-4_22.

[8] Carlo Blundo, Alfredo De Santis, Amir Herzberg, Shay Kutten, Ugo Vaccaro, and Moti Yung. "Perfectly-Secure Key Distribution for Dynamic Conferences". In: *CRYPTO'92*. Ed. by Ernest F. Brickell. Vol. 740. LNCS. Springer, Berlin, Heidelberg, Aug. 1993, pp. 471–486. DOI: 10.1007/3-540-48071-4_33.

[9] Dan Boneh and Matthew K. Franklin. "Identity-Based Encryption from the Weil Pairing". In: *CRYPTO 2001*. Ed. by Joe Kilian. Vol. 2139. LNCS. Springer, Berlin, Heidelberg, Aug. 2001, pp. 213–229. DOI: 10.1007/3-540-44647-8_13.

[10] Dan Boneh, Sam Kim, and Hart William Montgomery. "Private Puncturable PRFs from Standard Lattice Assumptions". In: *EUROCRYPT 2017, Part I*. Ed. by Jean-Sébastien Coron and Jesper Buus Nielsen. Vol. 10210. LNCS. Springer, Cham, 2017, pp. 415–445. DOI: 10.1007/978-3-319-56620-7_15.

[11] Dan Boneh, Sam Kim, and David J. Wu. "Constrained Keys for Invertible Pseudorandom Functions". In: *TCC 2017, Part I*. Ed. by Yael Kalai and Leonid Reyzin. Vol. 10677. LNCS. Springer, Cham, Nov. 2017, pp. 237–263. DOI: 10.1007/978-3-319-70500-2_9.

[12] Dan Boneh, Kevin Lewi, and David J. Wu. "Constraining Pseudorandom Functions Privately". In: *PKC 2017, Part II*. Ed. by Serge Fehr. Vol. 10175. LNCS. Springer, Berlin, Heidelberg, Mar. 2017, pp. 494–524. DOI: 10.1007/978-3-662-54388-7_17.

[13] Dan Boneh, Periklis A. Papakonstantinou, Charles Rackoff, Yevgeniy Vahlis, and Brent Waters. "On the Impossibility of Basing Identity Based Encryption on Trapdoor Permutations". In: *49th FOCS*. IEEE Computer Society Press, Oct. 2008, pp. 283–292. DOI: 10.1109/FOCS.2008.67.

[14] Dan Boneh and Brent Waters. "Constrained Pseudorandom Functions and Their Applications". In: *ASIACRYPT 2013, Part II*. Ed. by Kazue Sako and Palash Sarkar. Vol. 8270. LNCS. Springer, Berlin, Heidelberg, Dec. 2013, pp. 280–300. DOI: 10.1007/978-3-642-42045-0_15.

[15] Dan Boneh and Mark Zhandry. "Multiparty Key Exchange, Efficient Traitor Tracing, and More from Indistinguishability Obfuscation". In: *CRYPTO 2014, Part I*. Ed. by Juan A. Garay and Rosario Gennaro. Vol. 8616. LNCS. Springer, Berlin, Heidelberg, Aug. 2014, pp. 480–499. DOI: 10.1007/978-3-662-44371-2_27.

[16] Elette Boyle, Shafi Goldwasser, and Ioana Ivan. "Functional Signatures and Pseudorandom Functions". In: *PKC 2014*. Ed. by Hugo Krawczyk. Vol. 8383.

LNCS. Springer, Berlin, Heidelberg, Mar. 2014, pp. 501–519. DOI: 10.1007/978-3-642-54631-0_29.

[17] Zvika Brakerski, Jonathan Katz, Gil Segev, and Arkady Yerukhimovich. "Limits on the Power of Zero-Knowledge Proofs in Cryptographic Constructions". In: *TCC 2011*. Ed. by Yuval Ishai. Vol. 6597. LNCS. Springer, Berlin, Heidelberg, Mar. 2011, pp. 559–578. DOI: 10.1007/978-3-642-19571-6_34.

[18] Zvika Brakerski, Rotem Tsabary, Vinod Vaikuntanathan, and Hoeteck Wee. "Private Constrained PRFs (and More) from LWE". In: *TCC 2017, Part I*. Ed. by Yael Kalai and Leonid Reyzin. Vol. 10677. LNCS. Springer, Cham, Nov. 2017, pp. 264–302. DOI: 10.1007/978-3-319-70500-2_10.

[19] Zvika Brakerski and Vinod Vaikuntanathan. "Constrained Key-Homomorphic PRFs from Standard Lattice Assumptions - Or: How to Secretly Embed a Circuit in Your PRF". In: *TCC 2015, Part II*. Ed. by Yevgeniy Dodis and Jesper Buus Nielsen. Vol. 9015. LNCS. Springer, Berlin, Heidelberg, Mar. 2015, pp. 1–30. DOI: 10.1007/978-3-662-46497-7_1.

[20] Nicholas Brandt, Miguel Cueto Noval, Christoph U. Günther, Akin Ünal, and Stella Wohnig. "Constrained Verifiable Random Functions Without Obfuscation and Friends". In: *TCC 2025, Part IV*. Ed. by Benny Applebaum and Huijia (Rachel) Lin. Vol. 16271. LNCS. Springer, Cham, Dec. 2025, pp. 478–511. DOI: 10.1007/978-3-032-12290-2_16.

[21] Ran Canetti and Yilei Chen. "Constraint-Hiding Constrained PRFs for NC^1 from LWE". In: *EUROCRYPT 2017, Part I*. Ed. by Jean-Sébastien Coron and Jesper Buus Nielsen. Vol. 10210. LNCS. Springer, Cham, 2017, pp. 446–476. DOI: 10.1007/978-3-319-56620-7_16.

[22] Suvradip Chakraborty, Dennis Hofheinz, and Roman Langrehr. "Non-interactive Key Exchange: New Notions, New Constructions, and Forward Security". In: *PKC 2025, Part II*. Ed. by Tibor Jager and Jiaxin Pan. Vol. 15675. LNCS. Springer, Cham, May 2025, pp. 203–236. DOI: 10.1007/978-3-031-91823-0_7.

[23] Nishanth Chandran, Srinivasan Raghuraman, and Dhinakaran Vinayagamurthy. *Constrained Pseudorandom Functions: Verifiable and Delegatable*. Cryptology ePrint Archive, Report 2014/522. 2014. URL: https://eprint.iacr.org/2014/522.

[24] Yu Chen, Qiong Huang, and Zongyang Zhang. "Sakai-Ohgishi-Kasahara Identity-Based Non-Interactive Key Exchange Revisited and More". In: *ACISP 14*. Ed. by Willy Susilo and Yi Mu. Vol. 8544. LNCS. Springer, Cham, July 2014, pp. 274–289. DOI: 10.1007/978-3-319-08344-5_18.

[25] Kaishuo Cheng and Joseph Jaeger. "Adaptive Security for Constrained PRFs". In: *CRYPTO 2025, Part V*. Ed. by Yael Tauman Kalai and Seny F. Kamara. Vol. 16004. LNCS. Springer, Cham, Aug. 2025, pp. 598–627. DOI: 10.1007/978-3-032-01901-1_19.

[26] Geoffroy Couteau, Pierre Meyer, Alain Passelègue, and Mahshid Riahinia. "Constrained Pseudorandom Functions from Homomorphic Secret Sharing". In: *EUROCRYPT 2023, Part III*. Ed. by Carmit Hazay and Martijn Stam. Vol. 14006. LNCS. Springer, Cham, Apr. 2023, pp. 194–224. DOI: 10.1007/978-3-031-30620-4_7.

[27] Pratish Datta. "Constrained pseudorandom functions from functional encryption". In: *Theoretical Computer Science* 809 (2020), pp. 137–170. ISSN: 0304-3975. DOI: https://doi.org/10.1016/j.tcs.2019.12.004.

[28] Pratish Datta, Ratna Dutta, and Sourav Mukhopadhyay. "Constrained Pseudorandom Functions for Unconstrained Inputs Revisited: Achieving Verifiability and Key Delegation". In: *PKC 2017, Part II*. Ed. by Serge Fehr.

Vol. 10175. LNCS. Springer, Berlin, Heidelberg, Mar. 2017, pp. 463–493. DOI: 10.1007/978-3-662-54388-7_16.

[29] Pratish Datta, Ratna Dutta, and Sourav Mukhopadhyay. "Constrained Pseudorandom Functions for Turing Machines Revisited: How to Achieve Verifiability and Key Delegation". In: *Algorithmica* 81.9 (2019), pp. 3245–3390. DOI: 10.1007/S00453-019-00576-7.

[30] Pratish Datta, Ratna Dutta, and Sourav Mukhopadhyay. "Functional Encryption for Inner Product with Full Function Privacy". In: *PKC 2016, Part I*. Ed. by Chen-Mou Cheng, Kai-Min Chung, Giuseppe Persiano, and Bo-Yin Yang. Vol. 9614. LNCS. Springer, Berlin, Heidelberg, Mar. 2016, pp. 164–195. DOI: 10.1007/978-3-662-49384-7_7.

[31] Alex Davidson, Shuichi Katsumata, Ryo Nishimaki, Shota Yamada, and Takashi Yamakawa. "Adaptively Secure Constrained Pseudorandom Functions in the Standard Model". In: *CRYPTO 2020, Part I*. Ed. by Daniele Micciancio and Thomas Ristenpart. Vol. 12170. LNCS. Springer, Cham, Aug. 2020, pp. 559–589. DOI: 10.1007/978-3-030-56784-2_19.

[32] Apoorvaa Deshpande, Venkata Koppula, and Brent Waters. "Constrained Pseudorandom Functions for Unconstrained Inputs". In: *EUROCRYPT 2016, Part II*. Ed. by Marc Fischlin and Jean-Sébastien Coron. Vol. 9666. LNCS. Springer, Berlin, Heidelberg, May 2016, pp. 124–153. DOI: 10.1007/978-3-662-49896-5_5.

[33] Nico Döttling and Sanjam Garg. "Identity-Based Encryption from the Diffie-Hellman Assumption". In: *CRYPTO 2017, Part I*. Ed. by Jonathan Katz and Hovav Shacham. Vol. 10401. LNCS. Springer, Cham, Aug. 2017, pp. 537–569. DOI: 10.1007/978-3-319-63688-7_18.

[34] Régis Dupont and Andreas Enge. "Provably secure non-interactive key distribution based on pairings". In: *Discrete Appl. Math.* 154.2 (2006), 270–276. ISSN: 0166-218X.

[35] Laurent Eschenauer and Virgil D. Gligor. "A Key-Management Scheme for Distributed Sensor Networks". In: *ACM CCS 2002*. Ed. by Vijayalakshmi Atluri. ACM Press, Nov. 2002, pp. 41–47. DOI: 10.1145/586110.586117.

[36] Eduarda S. V. Freire, Dennis Hofheinz, Kenneth G. Paterson, and Christoph Striecks. "Programmable Hash Functions in the Multilinear Setting". In: *CRYPTO 2013, Part I*. Ed. by Ran Canetti and Juan A. Garay. Vol. 8042. LNCS. Springer, Berlin, Heidelberg, Aug. 2013, pp. 513–530. DOI: 10.1007/978-3-642-40041-4_28.

[37] Georg Fuchsbauer. "Constrained Verifiable Random Functions". In: *SCN 14*. Ed. by Michel Abdalla and Roberto De Prisco. Vol. 8642. LNCS. Springer, Cham, Sept. 2014, pp. 95–114. DOI: 10.1007/978-3-319-10879-7_7.

[38] Georg Fuchsbauer, Momchil Konstantinov, Krzysztof Pietrzak, and Vanishree Rao. "Adaptive Security of Constrained PRFs". In: *ASIACRYPT 2014, Part II*. Ed. by Palash Sarkar and Tetsu Iwata. Vol. 8874. LNCS. Springer, Berlin, Heidelberg, Dec. 2014, pp. 82–101. DOI: 10.1007/978-3-662-45608-8_5.

[39] Rosario Gennaro, Shai Halevi, Hugo Krawczyk, Tal Rabin, Steffen Reidt, and Stephen D. Wolthusen. "Strongly-Resilient and Non-interactive Hierarchical Key-Agreement in MANETs". In: *ESORICS 2008*. Ed. by Sushil Jajodia and Javier López. Vol. 5283. LNCS. Springer, Berlin, Heidelberg, Oct. 2008, pp. 49–65. DOI: 10.1007/978-3-540-88313-5_4.

[40] Oded Goldreich, Shafi Goldwasser, and Silvio Micali. "How to Construct Random Functions (Extended Abstract)". In: *25th FOCS*. IEEE Computer Society Press, Oct. 1984, pp. 464–479. DOI: 10.1109/SFCS.1984.715949.

[41] Vipul Goyal, Virendra Kumar, Satyanarayana V. Lokam, and Mohammad Mahmoody. "On Black-Box Reductions between Predicate Encryption Schemes". In: *TCC 2012*. Ed. by Ronald Cramer. Vol. 7194. LNCS. Springer, Berlin, Heidelberg, Mar. 2012, pp. 440–457. DOI: 10.1007/978-3-642-28914-9_25.

[42] Mohammad Hajiabadi, Roman Langrehr, Adam O'Neill, and Mingyuan Wang. "On the Black-Box Complexity of Private-Key Inner-Product Functional Encryption". In: *TCC 2024, Part III*. Ed. by Elette Boyle and Mohammad Mahmoody. Vol. 15366. LNCS. Springer, Cham, Dec. 2024, pp. 318–343. DOI: 10.1007/978-3-031-78020-2_11.

[43] Dennis Hofheinz. *Fully secure constrained pseudorandom functions using random oracles*. Cryptology ePrint Archive, Report 2014/372. 2014. URL: https://eprint.iacr.org/2014/372.

[44] Dennis Hofheinz, Tibor Jager, Dakshita Khurana, Amit Sahai, Brent Waters, and Mark Zhandry. "How to Generate and Use Universal Samplers". In: *ASIACRYPT 2016, Part II*. Ed. by Jung Hee Cheon and Tsuyoshi Takagi. Vol. 10032. LNCS. Springer, Berlin, Heidelberg, Dec. 2016, pp. 715–744. DOI: 10.1007/978-3-662-53890-6_24.

[45] Dennis Hofheinz, Akshay Kamath, Venkata Koppula, and Brent Waters. "Adaptively Secure Constrained Pseudorandom Functions". In: *FC 2019*. Ed. by Ian Goldberg and Tyler Moore. Vol. 11598. LNCS. Springer, Cham, Feb. 2019, pp. 357–376. DOI: 10.1007/978-3-030-32101-7_22.

[46] Dennis Hofheinz, Julia Kastner, and Karen Klein. "The Power of Undirected Rewindings for Adaptive Security". In: *CRYPTO 2023, Part II*. Ed. by Helena Handschuh and Anna Lysyanskaya. Vol. 14082. LNCS. Springer, Cham, Aug. 2023, pp. 725–758. DOI: 10.1007/978-3-031-38545-2_24.

[47] Jeremy Horwitz and Ben Lynn. "Toward Hierarchical Identity-Based Encryption". In: *EUROCRYPT 2002*. Ed. by Lars R. Knudsen. Vol. 2332. LNCS. Springer, Berlin, Heidelberg, 2002, pp. 466–481. DOI: 10.1007/3-540-46035-7_31.

[48] Russell Impagliazzo and Steven Rudich. "Limits on the Provable Consequences of One-Way Permutations". In: *21st ACM STOC*. ACM Press, May 1989, pp. 44–61. DOI: 10.1145/73007.73012.

[49] Chethan Kamath, Karen Klein, Krzysztof Pietrzak, and Michael Walter. "The Cost of Adaptivity in Security Games on Graphs". In: *TCC 2021, Part II*. Ed. by Kobbi Nissim and Brent Waters. Vol. 13043. LNCS. Springer, Cham, Nov. 2021, pp. 550–581. DOI: 10.1007/978-3-030-90453-1_19.

[50] Aggelos Kiayias, Stavros Papadopoulos, Nikos Triandopoulos, and Thomas Zacharias. "Delegatable pseudorandom functions and applications". In: *ACM CCS 2013*. Ed. by Ahmad-Reza Sadeghi, Virgil D. Gligor, and Moti Yung. ACM Press, Nov. 2013, pp. 669–684. DOI: 10.1145/2508859.2516668.

[51] Dennis Kügler and Markus Maurer. *A note on the weakness of the Maurer-Yacobi squaring method*. Tech. rep. TI 15 99. https://d-nb.info/958651418. Technical University of Darmstadt, Department of Computer Science, 1999.

[52] Roman Langrehr. "Non-interactive key exchange for many users". Doctoral Thesis. Zurich: ETH Zurich, 2025. DOI: 10.3929/ethz-b-000740644.

[53] Ueli M. Maurer and Yacov Yacobi. "A Non-interactive Public-Key Distribution System". In: *DCC* 9.3 (1996), pp. 305–316. DOI: 10.1023/A:1027332606155.

[54] Ueli M. Maurer and Yacov Yacobi. "A Remark on a Non-interactive Public-Key Distribution System (Rump Session)". In: *EUROCRYPT'92*. Ed. by Rainer A. Rueppel. Vol. 658. LNCS. Springer, Berlin, Heidelberg, May 1993, pp. 458–460. DOI: 10.1007/3-540-47555-9_39.

[55] Ueli M. Maurer and Yacov Yacobi. "Non-interactive Public-Key Cryptography". In: *EUROCRYPT'91*. Ed. by Donald W. Davies. Vol. 547. LNCS. Springer, Berlin, Heidelberg, Apr. 1991, pp. 498–507. DOI: 10.1007/3-540-46416-6_43.

[56] Ralph C. Merkle. "Secure communications over insecure channels". In: 21.4 (Apr. 1978), 294–299. ISSN: 0001-0782. DOI: 10.1145/359460.359473.

[57] Yasuyuki Murakami and Masao Kasahara. *Murakami-Kasahara ID-based Key Sharing Scheme Revisited —In Comparison with Maurer-Yacobi Schemes—*. Cryptology ePrint Archive, Report 2005/306. 2005. URL: https://eprint.iacr.org/2005/306.

[58] Periklis A. Papakonstantinou, Charles W. Rackoff, and Yevgeniy Vahlis. *How powerful are the DDH hard groups?* Cryptology ePrint Archive, Report 2012/653. 2012. URL: https://eprint.iacr.org/2012/653.

[59] Kenneth G. Paterson and Sriramkrishnan Srinivasan. "On the relations between non-interactive key distribution, identity-based encryption and trapdoor discrete log groups". In: *DCC* 52.2 (2009), pp. 219–241. DOI: 10.1007/s10623-009-9278-y.

[60] Chris Peikert and Sina Shiehian. "Privately Constraining and Programming PRFs, the LWE Way". In: *PKC 2018, Part II*. Ed. by Michel Abdalla and Ricardo Dahab. Vol. 10770. LNCS. Springer, Cham, Mar. 2018, pp. 675–701. DOI: 10.1007/978-3-319-76581-5_23.

[61] Naty Peter, Rotem Tsabary, and Hoeteck Wee. "One-One Constrained Pseudorandom Functions". In: *ITC 2020*. Ed. by Yael Tauman Kalai, Adam D. Smith, and Daniel Wichs. Vol. 163. LIPIcs. Schloss Dagstuhl, June 2020, 13:1–13:22. DOI: 10.4230/LIPIcs.ITC.2020.13.

[62] Mahalingam Ramkumar, Nasir Memon, and Rahul Simha. "A hierarchical key pre-distribution scheme". In: *2005 IEEE International Conference on Electro Information Technology*. 2005. DOI: 10.1109/EIT.2005.1626994.

[63] Omer Reingold, Luca Trevisan, and Salil P. Vadhan. "Notions of Reducibility between Cryptographic Primitives". In: *TCC 2004*. Ed. by Moni Naor. Vol. 2951. LNCS. Springer, Berlin, Heidelberg, Feb. 2004, pp. 1–20. DOI: 10.1007/978-3-540-24638-1_1.

[64] Amit Sahai and Brent Waters. "How to use indistinguishability obfuscation: deniable encryption, and more". In: *46th ACM STOC*. Ed. by David B. Shmoys. ACM Press, 2014, pp. 475–484. DOI: 10.1145/2591796.2591825.

[65] Ryuichi Sakai, Kiyoshi Ohgishi, and Masao Kasahara. "Cryptosystems based on Pairing". In: *SCIS 2000*. Okinawa, Japan, Jan. 2000.

[66] Gili Schul-Ganz and Gil Segev. "Generic-Group Identity-Based Encryption: A Tight Impossibility Result". In: *ITC 2021*. Ed. by Stefano Tessaro. Vol. 199. LIPIcs. Schloss Dagstuhl, July 2021, 26:1–26:23. DOI: 10.4230/LIPIcs.ITC.2021.26.

[67] Sacha Servan-Schreiber. "Constrained Pseudorandom Functions for Inner-Product Predicates from Weaker Assumptions". In: *ASIACRYPT 2024, Part II*. Ed. by Kai-Min Chung and Yu Sasaki. Vol. 15485. LNCS. Springer, Singapore, Dec. 2024, pp. 232–265. DOI: 10.1007/978-981-96-0888-1_8.

[68] Adi Shamir. "Identity-Based Cryptosystems and Signature Schemes". In: *CRYPTO'84*. Ed. by G. R. Blakley and David Chaum. Vol. 196. LNCS. Springer, Berlin, Heidelberg, Aug. 1984, pp. 47–53. DOI: 10.1007/3-540-39568-7_5.

[69] Elaine Shi and Brent Waters. "Delegating Capabilities in Predicate Encryption Systems". In: *ICALP 2008, Part II*. Ed. by Luca Aceto, Ivan Damgård, Leslie Ann Goldberg, Magnús M. Halldórsson, Anna Ingólfsdóttir, and Igor Walukiewicz.

Vol. 5126. LNCS. Springer, Berlin, Heidelberg, July 2008, pp. 560–578. DOI: 10.1007/978-3-540-70583-3_46.

[70] Mark Zhandry. "To Label, or Not To Label (in Generic Groups)". In: *CRYPTO 2022, Part III*. Ed. by Yevgeniy Dodis and Thomas Shrimpton. Vol. 13509. LNCS. Springer, Cham, Aug. 2022, pp. 66–96. DOI: 10.1007/978-3-031-15982-4_3.

A Identity-based NIKE

We recall the definition of ID-NIKE from [34, 59]. For simplicity, our definition has no public parameters (or master public key), since such public information can simply be stored in the master secret key and every user secret key. Moreover, we generalize the definition to N-party ID-NIKE.

Definition 15 (ID-NIKE). *An N-party identity-based non-interactive key exchange (ID-NIKE) scheme with identity space $\mathcal{ID}$ and shared key space $\mathcal{K}$ consists of three PPT algorithms* $\mathsf{ID\text{-}NIKE} = (\mathsf{MasterKeyGen}, \mathsf{USKGen}, \mathsf{SharedKey})$ *where*

- $\mathsf{MasterKeyGen}(1^\lambda)$ *inputs the unary encoded security parameter 1^λ and outputs a master secret key* msk.
- $\mathsf{USKGen}(\mathsf{msk}, \mathsf{id} \in \mathcal{ID})$ *generates user secret keys from the master secret key. It inputs a master secret key* msk *and an identity* $\mathsf{id} \in \mathcal{ID}$. *It outputs a user secret key* $\mathsf{usk}[\![\mathsf{id}]\!]$.
- $\mathsf{SharedKey}(S \subseteq \mathcal{ID}, \mathsf{usk}[\![\mathsf{id}]\!])$ *computes the shared keys. It is deterministic and inputs a set of identities S with $|S| = N$ and and a user secret key* $\mathsf{usk}[\![\mathsf{id}]\!]$ *for an identity* $\mathsf{id} \in S$. *It outputs a shared key $k \in \mathcal{K}$.*

In the following we use ID-NIKE as shorthand for 2-party ID-NIKE and use the notation $\mathsf{SharedKey}(\mathsf{usk}[\![\mathsf{id}_1]\!], \mathsf{id}_2) := \mathsf{SharedKey}(\{\mathsf{id}_1, \mathsf{id}_2\}, \mathsf{usk}[\![\mathsf{id}_1]\!])$ for $\mathsf{id}_1, \mathsf{id}_2 \in \mathcal{ID}$ with $\mathsf{id}_1 \neq \mathsf{id}_2$.

Definition 16 (Correctness). *An N-party ID-NIKE is correct if for all $S \subseteq \mathcal{ID}$ with $|S| = N$ and $\mathsf{id}_1, \mathsf{id}_2 \in S$ we have*

$$\Pr[\mathsf{SharedKey}(S, \mathsf{usk}[\![\mathsf{id}_1]\!]) = \mathsf{SharedKey}(S, \mathsf{usk}[\![\mathsf{id}_2]\!])] \geq 1 - \mathsf{negl}(\lambda)$$

for a negligible function negl, *where the probability is taken over* $\mathsf{msk} \leftarrow \mathsf{MasterKeyGen}(1^\lambda)$, $\mathsf{usk}[\![\mathsf{id}_1]\!] \leftarrow \mathsf{USKGen}(\mathsf{msk}, \mathsf{id}_1)$, *and* $\mathsf{usk}[\![\mathsf{id}_2]\!] \leftarrow \mathsf{USKGen}(\mathsf{msk}, \mathsf{id}_2)$.

We recall the standard adaptive security notion for ID-NIKE from [59] which strengthens [34], adapted to N-party ID-NIKE.

Definition 17 (Security). *An N-party ID-NIKE* $\mathsf{ID\text{-}NIKE}$ *is* adaptively secure *iff for every adversary $\mathcal{A}$*

$$\mathsf{Adv}^{\mathsf{adpt}}_{\mathsf{ID\text{-}NIKE},\mathcal{A}}(\lambda) := \left| \Pr[\mathsf{Exp}^{\mathsf{adpt}}_{\mathsf{ID\text{-}NIKE},\mathcal{A}}(\lambda) \Rightarrow 1] - \frac{1}{2} \right| \leq \mathsf{negl}(\lambda),$$

for a negligible function negl. *The game* $\mathsf{Exp}^{\mathsf{adpt}}_{\mathsf{ID\text{-}NIKE},\mathcal{A}}(\lambda)$ *is defined in Figure 3.*

B Equivalence of 2-party ID-NIKE and L/R constrained PRFs

For this and the following section the following two constraint classes are relevant:

$\underline{\mathsf{Exp}^{\mathsf{adpt}}_{\mathsf{ID\text{-}NIKE},\mathcal{A}}(\lambda)}$:
$\mathsf{msk} \leftarrow \mathsf{MasterKeyGen}(1^\lambda)$
$b \xleftarrow{\$} \{0,1\}$
$R_U \leftarrow \emptyset$
$R_K \leftarrow \emptyset$
$b' \leftarrow \mathcal{A}^{\textsc{Extract}(\cdot),\textsc{Reveal}(\cdot,\cdot),\textsc{Chal}(\cdot)}(1^\lambda)$
if $\exists \mathsf{id} \in S^\star : \mathsf{id} \in R_U \vee S^\star \in R_K$ **then**
  return $b'' \xleftarrow{\$} \{0,1\}$
else
  return $b \stackrel{?}{=} b'$

$\underline{\textsc{Extract}(\mathsf{id})}$:
$R_U \leftarrow R_U \cup \{\mathsf{id}\}$
return $\mathsf{USKGen}(\mathsf{msk}, \mathsf{id})$

$\underline{\textsc{Reveal}(S \subseteq \mathcal{ID}, \mathsf{id})}$:
if $|S| \neq N \vee \mathsf{id} \notin S$ **then return** $\perp$
$R_K \leftarrow R_K \cup \{S\}$
$\mathsf{usk}[\![\mathsf{id}]\!] \xleftarrow{\$} \mathsf{USKGen}(\mathsf{msk}, \mathsf{id})$
return $\mathsf{SharedKey}(S, \mathsf{usk}[\![\mathsf{id}]\!])$

$\underline{\textsc{Chal}(S^\star, \mathsf{id}^\star)}$: //only one query
if $|S^\star| \neq N \vee \mathsf{id}^\star \notin S^\star$ **then return** $\perp$
if $b = 0$ **then**
  $\mathsf{usk}[\![\mathsf{id}^\star]\!] \xleftarrow{\$} \mathsf{USKGen}(\mathsf{msk}, \mathsf{id}^\star)$
  return $\mathsf{SharedKey}(S^\star, \mathsf{usk}[\![\mathsf{id}^\star]\!])$
else
  return $k^\star \xleftarrow{\$} \mathcal{K}$

Fig. 3. The adaptive security experiment for ID-NIKE.

Definition 18 (One out of N block fixing CPRF). *For $N \in \mathbb{N}$ with $N \geq 2$ a* one out of N block fixing CPRF *is a CPRF for the domain $\mathcal{X}_\lambda := (\{0,1\}^\lambda)^N$ and the constraint class*

$$\mathcal{C}_\lambda := \{[i:x] \mid i \in [N], x \in \{0,1\}^\lambda\} \text{ where}$$
$$[i:x] := \{(a_1, \ldots, a_N) \in (\{0,1\}^\lambda)^N \mid a_i = x\} \text{ for } i \in [N], x \in \{0,1\}^\lambda$$

Definition 19 (Left/Right CPRF). *A* left/right CPRF *(L/R CPRF) is a one out of two block fixing CPRF.*

For left/right CPRF we use the following notation

$$(a, *) := \{(a, b) \mid b \in \{0,1\}^\lambda\} \text{ for } a \in \{0,1\}^\lambda$$
$$(*, b) := \{(a, b) \mid a \in \{0,1\}^\lambda\} \text{ for } b \in \{0,1\}^\lambda$$

Boneh and Waters [14, Section 6.2] proved that L/R CPRFs imply (2-party) identity-based non-interactive key exchange. In this section we show that the reverse is also true.

We present the transformation from an ID-NIKE to a L/R CPRF in Figure 4. The Eval algorithm of this PRF for the unconstrained key is probabilistic for simplicity. We can make it deterministic by picking the randomness from a regular PRF whose key is part of the CPRF key. Since regular PRFs can be built from one-way functions (OWFs) and ID-NIKE implies OWFs, this does not change the result.

Theorem 10. *If* ID-NIKE *is a correct ID-NIKE, then* CPRF[ID-NIKE] *is a correct CPRF for the left/right constraint class.*

$\underline{\mathsf{KeyGen}(1^\lambda)\text{:}}$
return $\mathsf{MasterKeyGen}(1^\lambda)$
 	
$\underline{\mathsf{Constrain}(K, C)\text{:}}$
if $C = (x, *)$ **then**
⌊**return** $\mathsf{USKGen}(\mathsf{msk}, (\mathsf{L}, x))$
if $C = (*, y)$ **then**
⌊**return** $\mathsf{USKGen}(\mathsf{msk}, (\mathsf{R}, y))$
	$\underline{\mathsf{Eval}(\mathsf{msk}, (x, y))\text{:}}$
	$\mathsf{usk}[\![(\mathsf{L}, x)]\!] \leftarrow \mathsf{USKGen}(\mathsf{msk}, (\mathsf{L}, x))$
	return $\mathsf{SharedKey}(\mathsf{usk}[\![(\mathsf{L}, x)]\!], (\mathsf{R}, y))$
	
	$\underline{\mathsf{Eval}(\mathsf{usk}[\![(\mathsf{L}, x)]\!], (x, y))\text{:}}$
	return $\mathsf{SharedKey}(\mathsf{usk}[\![(\mathsf{L}, x)]\!], (\mathsf{R}, y))$
	
	$\underline{\mathsf{Eval}(\mathsf{usk}[\![(\mathsf{R}, y)]\!], (x, y))\text{:}}$
	return $\mathsf{SharedKey}(\mathsf{usk}[\![(\mathsf{R}, y)]\!], (\mathsf{L}, x))$

Fig. 4. The construction of a L/R constrained PRF $\mathsf{CPRF}[\mathsf{ID\text{-}NIKE}] = (\mathsf{KeyGen}, \mathsf{Constrain}, \mathsf{Eval})$ from an ID-NIKE $\mathsf{ID\text{-}NIKE} = (\mathsf{MasterKeyGen}, \mathsf{USKGen}, \mathsf{SharedKey})$ with identity space $\mathcal{ID} = \{\mathsf{L}, \mathsf{R}\} \times \{0,1\}^\lambda$ and key space $\mathcal{K} = \mathcal{Y}_\lambda$.

Proof. Let $(x, y) \in \mathcal{X}_\lambda$, $\mathsf{msk} \leftarrow \mathsf{MasterKeyGen}(1^\lambda)$ a secret key for $\mathsf{CPRF}[\mathsf{ID\text{-}NIKE}]$ and $\mathsf{usk}[\![(\mathsf{L}, x)]\!] \leftarrow \mathsf{USKGen}(\mathsf{msk}, (\mathsf{L}, x))$ and $\mathsf{usk}[\![(\mathsf{R}, y)]\!] \leftarrow \mathsf{Eval}(\mathsf{usk}[\![(\mathsf{R}, y)]\!]$. These three types of keys are the only key types that allow you to evaluate the PRF on (x, y) for this constraint class. Since

$$\mathsf{SharedKey}(\mathsf{usk}[\![(\mathsf{L}, x)]\!], (\mathsf{R}, y)) = \mathsf{SharedKey}(\mathsf{usk}[\![(\mathsf{R}, y)]\!], (\mathsf{L}, x))$$

by correctness of $\mathsf{ID\text{-}NIKE}$, all three evaluate to the same random value. □

Theorem 11. *If* $\mathsf{ID\text{-}NIKE}$ *is adaptively secure, then* $\mathsf{CPRF}[\mathsf{ID\text{-}NIKE}]$ *is adaptively pseudorandom. More precisely, for every adversary* $\mathcal{A}$ *there exists an adversary* $\mathcal{B}$ *with*

$$\mathsf{Adv}^{\mathsf{adpt\text{-}pr}}_{\mathsf{CPRF}[\mathsf{ID\text{-}NIKE}],\mathcal{A}}(\lambda) \leq \mathsf{Adv}^{\mathsf{adpt}}_{\mathsf{ID\text{-}NIKE},\mathcal{B}}(\lambda).$$

Proof. The reduction $\mathcal{B}$ runs the adversary $\mathcal{A}$ and simulates every $\textsc{Eval}((x, y))$ query by making a $\textsc{Reveal}((\mathsf{L}, x), (\mathsf{R}, y))$ query, each $\textsc{Constrain}((x, *))$ query by a $\textsc{Extract}((\mathsf{L}, x))$ query, each each $\textsc{Constrain}((*, y))$ query by a $\textsc{Extract}((\mathsf{R}, y))$ query, and the $\textsc{Chal}((x^\star, y^\star)$ query by a $\textsc{Chal}((\mathsf{L}, x^\star), (\mathsf{R}, y^\star))$ query. It is easy too see that this simulates the game for the adversary $\mathcal{A}$ perfectly.

We need to show that the non-trivial winning condition of the adaptive security game for $\mathsf{ID\text{-}NIKE}$ is satisfied if the adversary $\mathcal{A}$ does not violate the non-trivial winning condition of the adaptive pseudorandomness game for $\mathsf{CPRF}[\mathsf{ID\text{-}NIKE}]$. Therefore, let $(x^\star, y^\star)$ be the input of $\mathcal{A}$'s challenge query. Then $\mathcal{B}$ makes the challenge query for the identities $(\mathsf{L}, x^\star)$ $(\mathsf{R}, y^\star)$. The only way $\mathcal{B}$ would make a $\textsc{Reveal}((\mathsf{L}, x^\star), (\mathsf{R}, y^\star))$ query is if $\mathcal{A}$ makes a $\textsc{Eval}((x^\star, y^\star))$ query, the only way $\mathcal{B}$ would make a $\textsc{Extract}((\mathsf{L}, x^\star))$ query is if $\mathcal{A}$ would make a $\textsc{Constrain}((x^\star, *))$ query, and finally the only way $\mathcal{B}$ would make a $\textsc{Extract}((\mathsf{R}, y^\star)$ query is if $\mathcal{A}$ would make a $\textsc{Constrain}((*, y^\star))$ query. These cases cover all queries that might cause $\mathcal{B}$ to violate the non-trivial winning condition and in each case we show that $\mathcal{A}$ makes a query violating the non-trivial winning condition of the adaptive pseudorandomness game for $\mathsf{CPRF}[\mathsf{ID\text{-}NIKE}]$. □

$\underline{\mathsf{MasterKeyGen}(1^\lambda)}$:	$\underline{\mathsf{SharedKey}(S, \mathsf{usk}[\![\mathsf{id}]\!])}$:
return $K \leftarrow \mathsf{KeyGen}(1^\lambda)$	**parse** $(K[\![[i : \mathsf{id}]]\!])_{1 \le i \le N} := \mathsf{usk}[\![\mathsf{id}]\!]$
$\underline{\mathsf{USKGen}(K, \mathsf{id})}$:	**let** $S = \{\mathsf{id}_1, \ldots, \mathsf{id}_N\}$ **such that** $\mathsf{id}_1 < \cdots < \mathsf{id}_N$
for $i \in \{1, \ldots, N\}$ **do**	**let** $i \in [N]$ **such that** $\mathsf{id}_i = \mathsf{id}$
$\lfloor K[\![[i : \mathsf{id}]]\!] := \mathsf{Constrain}(K, [i : \mathsf{id}])$	**return** $\mathsf{Eval}(K[\![[i : \mathsf{id}]]\!], (\mathsf{id}_1, \ldots, \mathsf{id}_N))$
return $\mathsf{usk}[\![\mathsf{id}]\!] = (K[\![[i : \mathsf{id}]]\!])_{1 \le i \le N}$	

Fig. 5. The transformation from a one out of N block fixing CPRFs $\mathsf{CPRF} = (\mathsf{KeyGen}, \mathsf{Constrain}, \mathsf{Eval})$ to a N-party ID-NIKE $\mathsf{ID\text{-}NIKE}[\mathsf{CPRF}] := (\mathsf{MasterKeyGen}, \mathsf{USKGen}, \mathsf{SharedKey})$ with identity space $\mathcal{ID} = \{0,1\}^\lambda$. Identities are compared according to their lexicographic order (or any other fixed total order).

C Equivalence of N-party ID-NIKE and one out of N block fixing constrained PRFs

In this section we generalize the equivalence between 2-party ID-NIKE and left/right CPRFs to N-party ID-NIKE and one out of N block fixing CPRFs.

C.1 From one out of N fixing constrained PRFs to N-party ID-NIKE

Hofheinz showed that a bit-fixing CPRF can be used to construct N-party ID-NIKE [43]. We recall their transformation in Figure 5, but adapt it to use one out of N block fixing CPRFs instead, which is better suited for our results.

Theorem 12. *If* CPRF *is correct CPRF, then* $\mathsf{CPRF}[\mathsf{ID\text{-}NIKE}]$ *is a correct* N*-party ID-NIKE.*

Proof. Let $K \leftarrow \mathsf{KeyGen}(1^\lambda)$, $S = \{\mathsf{id}_1, \ldots, \mathsf{id}_N\} \subseteq \mathcal{ID}$ with $\mathsf{id}_1 < \cdots < \mathsf{id}_N$, $i, j \in [N]$ and $\mathsf{usk}[\![\mathsf{id}_i]\!] \leftarrow \mathsf{USKGen}(K, \mathsf{id}_i), \mathsf{usk}[\![\mathsf{id}_j]\!] \leftarrow \mathsf{USKGen}(K, \mathsf{id}_j)$ containing $K[\![[i : \mathsf{id}_i]]\!] := \mathsf{Constrain}(K, [i : \mathsf{id}_i]), K[\![[j : \mathsf{id}_j]]\!] := \mathsf{Constrain}(K, [j : \mathsf{id}_j])$. Then, since CPRF is correct,

$$\begin{aligned}\mathsf{SharedKey}(S, \mathsf{usk}[\![\mathsf{id}_i]\!]) &= \mathsf{Eval}(K[\![[i : \mathsf{id}_i]]\!], (\mathsf{id}_1, \ldots, \mathsf{id}_N)) \\ &= \mathsf{Eval}(K[\![[j : \mathsf{id}_j]]\!], (\mathsf{id}_1, \ldots, \mathsf{id}_N)) = \mathsf{SharedKey}(S, \mathsf{usk}[\![\mathsf{id}_j]\!]).\end{aligned}$$

□

Theorem 13. *If* CPRF *is an adaptively pseudorandom and correct CPRF, then* $\mathsf{ID\text{-}NIKE}[\mathsf{CPRF}]$ *is adaptively secure. More precisely, for every adversary* $\mathcal{A}$ *there exists an adversary* $\mathcal{B}$ *with*

$$\mathsf{Adv}^{\mathsf{adpt}}_{\mathsf{ID\text{-}NIKE}[\mathsf{CPRF}],\mathcal{A}}(\lambda) \le \mathsf{Adv}^{\mathsf{adpt\text{-}pr}}_{\mathsf{CPRF},\mathcal{B}}(\lambda).$$

$\underline{\mathsf{KeyGen}(1^\lambda)}$: **return** $\mathsf{MasterKeyGen}(1^\lambda)$ $\underline{\mathsf{Constrain}(K, [i:x])}$: **return** $\mathsf{USKGen}(\mathsf{msk}, (i,x))$	$\underline{\mathsf{Eval}(\mathsf{msk}, (x_1, \dots, x_N))}$: $\mathsf{usk}[\![(1,x_1)]\!] \leftarrow \mathsf{USKGen}(\mathsf{msk}, (1,x_1))$ **return** $\mathsf{SharedKey}(\{(1,x_1), \dots, (N,x_N)\}, \mathsf{usk}[\![(1,x_1)]\!])$ $\underline{\mathsf{Eval}(\mathsf{usk}[\![(i,x_i)]\!], (x_1, \dots, x_N))}$: **return** $\mathsf{SharedKey}(\{(1,x_1), \dots, (N,x_N)\}, \mathsf{usk}[\![(i,x_i)]\!])$

Fig. 6. The construction of an one out of N block fixing constrained PRF $\mathsf{CPRF}[\mathsf{ID\text{-}NIKE}] = (\mathsf{KeyGen}, \mathsf{Constrain}, \mathsf{Eval})$ from an N-party ID-NIKE $\mathsf{ID\text{-}NIKE} = (\mathsf{MasterKeyGen}, \mathsf{USKGen}, \mathsf{SharedKey})$ with identity space $\mathcal{ID} = [N] \times \{0,1\}^\lambda$ and key space $\mathcal{K} = \mathcal{Y}_\lambda$.

Proof. We describe a direct reduction. The reduction forwards its input 1^λ to the adversary.

For each $\textsc{Extract}(\mathsf{id})$ oracle call of the adversary the reduction calls $K[\![[i : \mathsf{id}]]\!] := \textsc{Constrain}([i : \mathsf{id}])$ and returns all their results. For each $\textsc{Reveal}(S \subseteq \mathcal{ID}, \mathsf{id})$ query with $S = \{\mathsf{id}_1, \dots, \mathsf{id}_N\}$ for $\mathsf{id}_1 < \dots < \mathsf{id}_N$ the reduction calls $\mathsf{Eval}((\mathsf{id}_1, \dots, \mathsf{id}_N))$ and returns the result. Since CPRF is correct, this perfectly simulates the $\textsc{Reveal}$ oracle. Similarly, the adversary's $\textsc{Chal}(S^\star)$ oracle query for $S^\star = \{\mathsf{id}_1^\star, \dots, \mathsf{id}_N^\star\}$ is simulated with a $\textsc{Chal}((\mathsf{id}_1^\star, \dots, \mathsf{id}_N^\star))$ query for the underlying CPRF.

It remains to show that the reduction only violates the non-trivial winning condition, if the adversary does so. On the one hand, the reduction can violate the non-trivial winning condition if it makes an $\mathsf{Eval}((\mathsf{id}_1^\star, \dots, \mathsf{id}_N^\star))$ query. However, this happens only if the adversary makes an $\textsc{Reveal}(\{\mathsf{id}_1^\star, \dots, \mathsf{id}_N^\star\}, \mathsf{id}_i^\star)$ query and thus it also violates its non-trivial winning condition. On the other hand, the reduction can violate its non-trivial winning condition if it makes a $K[\![[i : \mathsf{id}_i^\star]]\!] := \textsc{Constrain}([i : \mathsf{id}_i^\star])$ query. However, this happens only when the adversary makes an $\textsc{Extract}(\mathsf{id}_i^\star)$ query and thus it also violates its non-trivial winning condition. □

C.2 From N-party ID-NIKE to one out of N block fixing constrained PRFs

In this section we show how a one out of N block fixing constrained PRFs can be constructed from an N-party ID-NIKE by generalizing the construction in Appendix B.

We present the transformation in Figure 6. The Eval algorithm of this PRF for the unconstrained key is probabilistic for simplicity. We can make it deterministic by picking the randomness from a regular PRF whose key is part of the CPRF key. Since regular PRFs can be built from one-way functions (OWFs) and ID-NIKE implies OWFs, this does not change the result.

Theorem 14. *If* ID-NIKE *is a correct N-party ID-NIKE, then* CPRF[ID-NIKE] *is a correct CPRF for the one out of N block fixing constraint class.*

Proof. Let $(x_1, \ldots, x_N) \in \mathcal{X}_\lambda$, $\mathsf{msk} \leftarrow \mathsf{MasterKeyGen}(1^\lambda)$ a secret key for CPRF[ID-NIKE] and $\mathsf{usk}[\![(i, x_i)]\!] \leftarrow \mathsf{USKGen}(\mathsf{msk}, (i, x_i))$ for $i \in [N]$. These keys are the only keys that allow you to evaluate the PRF on $(x_1, \ldots, x_N)$ for this constraint class. Since

$$\begin{aligned}\mathsf{SharedKey}(\{(1, x_1), \ldots, (N, x_N)\}, \mathsf{usk}[\![(i, x_i)]\!]) \\ = \mathsf{SharedKey}(\{(1, x_1), \ldots, (N, x_N)\}, \mathsf{usk}[\![(1, x_1)]\!])\end{aligned}$$

by correctness of ID-NIKE, all keys evaluate to the same random value. □

Theorem 15. *If* ID-NIKE *is an adaptively secure N-party ID-NIKE, then* CPRF[ID-NIKE] *is an adaptively pseudorandom CPRF. More precisely, for every adversary $\mathcal{A}$ there exists an adversary $\mathcal{B}$ with*

$$\mathsf{Adv}^{\mathsf{adpt\text{-}pr}}_{\mathsf{CPRF[ID\text{-}NIKE]},\mathcal{A}}(\lambda) \leq \mathsf{Adv}^{\mathsf{adpt}}_{\mathsf{ID\text{-}NIKE},\mathcal{B}}(\lambda).$$

Proof. The reduction runs the adversary and simulates every $\textsc{Eval}((x_1, \ldots, x_N))$ query by making a $\textsc{Reveal}(\{(1, x_1), \ldots, (N, x_N)\}, (1, x_1))$ query, each $\textsc{Constrain}([i : x])$ query by a $\textsc{Extract}((i, x))$ query, and the $\textsc{Chal}((x_1^\star, \ldots, x_N^\star))$ query by a $\textsc{Chal}(\{(1, x_1^\star), \ldots, (N, x_N^\star)\}, (1, x_1^\star))$ query. It is easy to see that this simulates the game for the adversary $\mathcal{A}$ perfectly.

We need to show that the non-trivial winning condition of the adaptive security game for ID-NIKE is satisfied if the adversary $\mathcal{A}$ does not violate the non-trivial winning condition of the adaptive pseudorandomness game for CPRF[ID-NIKE]. Therefore, let $(x_1^\star, \ldots, x_N^\star)$ be the input of $\mathcal{A}$'s challenge query. The reduction makes the challenge query for the identity set $\{(1, x_1^\star), \ldots, (N, x_N^\star)\}$. The only way $\mathcal{B}$ would make a $\textsc{Reveal}(\{(1, x_1^\star), \ldots, (N, x_N^\star)\}, (i, x_i^\star))$ query is if $\mathcal{A}$ makes a $\textsc{Eval}(\{(1, x_1^\star), \ldots, (N, x_N^\star)\})$ query and the only way the reduction would make a $\textsc{Extract}((i, x_i^\star))$ query is if $\mathcal{A}$ would make a $\textsc{Constrain}([i : x])$ query. These cases cover all queries that might cause the reduction to violate the non-trivial winning condition and in each case we show that $\mathcal{A}$ makes a query violating the non-trivial winning condition of the adaptive pseudorandomness game for CPRF[ID-NIKE]. □

D Extension to Hierarchical identity-based NIKE

D.1 Definition

In this section we introduce the notion of hierarchical identity-based NIKE (HID-NIKE), which generalizes identity-based NIKE analogously to how hierarchical identity-based encryption (HIBE) [47] generalizes identity-based encryption (IBE) [68, 9]. A similar notion has been considered in [8, 39] and in [22] in the multi-authority setting.

Definition 20 (HID-NIKE). *An L-level N-party* hierarchical *identity-based non-interactive key exchange (HID-NIKE) scheme with identity base set $\mathcal{ID}$ and shared key space $\mathcal{K}$ consists of four PPT algorithms* $\mathsf{HID\text{-}NIKE} = (\mathsf{MasterKeyGen}, \mathsf{USKGen}, \mathsf{USKDel}, \mathsf{SharedKey})$ *where*

- $(\mathsf{MasterKeyGen}, \mathsf{USKGen}, \mathsf{SharedKey})$ *is an N-party ID-NIKE for identity space $\mathcal{ID}^{\leq L}$ (see Definition 15).*
- $\mathsf{USKDel}(\mathsf{usk}[\![\mathsf{id}]\!], \mathsf{id}' \in \mathcal{ID})$ *generates a user secret key from a user secret key for parent's identity. It inputs a user secret key* $\mathsf{usk}[\![\mathsf{id}]\!]$ *for* $\mathsf{id}^{\leq L-1}$ *and an identity element* $\mathsf{id}' \in \mathcal{ID}$*. It outputs a user secret key* $\mathsf{usk}[\![\mathsf{id}\|(\mathsf{id}')]\!]$*.*

The following definition is adapted from [22].

Definition 21 (Delegation invariance). *An L-level N-party HID-NIKE* $\mathsf{HID\text{-}NIKE} = (\mathsf{MasterKeyGen}, \mathsf{USKGen}, \mathsf{USKDel}, \mathsf{SharedKey})$ *with identity base set $\mathcal{ID}$ is* delegation invariant *iff*

- $\forall \mathsf{id} \in \mathcal{ID}^{\leq L-1}, \mathsf{id}' \in \mathcal{ID}$,
- $\mathsf{msk} \leftarrow \mathsf{MasterKeyGen}(1^\lambda)$,
- $\mathsf{usk}[\![\mathsf{id}\|(\mathsf{id}')]\!] \leftarrow \mathsf{USKGen}(\mathsf{msk}, \mathsf{id}\|(\mathsf{id}'))$,
- $\mathsf{usk}[\![\mathsf{id}]\!] \leftarrow \mathsf{USKGen}(\mathsf{msk}, \mathsf{id})$, *and*
- $\widehat{\mathsf{usk}}[\mathsf{id}\|(\mathsf{id}')] \leftarrow \mathsf{USKDel}(\mathsf{usk}[\![\mathsf{id}]\!], \mathsf{id}')$, *we have*

$$(\mathsf{msk}, \mathsf{usk}[\![\mathsf{id}]\!], \mathsf{usk}[\![\mathsf{id}\|(\mathsf{id}')]\!]) \approx_s (\mathsf{msk}, \mathsf{usk}[\![\mathsf{id}]\!], \widehat{\mathsf{usk}}[\mathsf{id}\|\mathsf{id}']),$$

where the probability is taken over the randomness used to sample the variables msk, $\mathsf{usk}[\![\mathsf{id}]\!]$, $\mathsf{usk}[\![\mathsf{id}\|(\mathsf{id}')]\!]$, *and* $\widehat{\mathsf{usk}}[\mathsf{id}\|(\mathsf{id}')]$*. If the HID-NIKE scheme is defined relative to an oracle (e.g. a random oracle), we need*

$$(\mathsf{msk}, \mathcal{O}, \mathsf{usk}[\![\mathsf{id}]\!], \mathsf{usk}[\![\mathsf{id}\|(\mathsf{id}')]\!]) \approx_s (\mathsf{msk}, \mathcal{O}, \mathsf{usk}[\![\mathsf{id}]\!], \widehat{\mathsf{usk}}[\mathsf{id}\|\mathsf{id}']),$$

where $\mathcal{O}$ is the entire function table of the oracle and all other variables are defined as above.

We say that an L-level N-party HID-NIKE is correct if its corresponding ID-NIKE $(\mathsf{MasterKeyGen}, \mathsf{USKGen}, \mathsf{SharedKey})$ is correct.

Definition 22 (Security). *A L-level N-party HID-NIKE* HID-NIKE *is* adaptively secure *iff for every adversary $\mathcal{A}$*

$$\mathsf{Adv}^{\mathsf{h\text{-}adpt}}_{\mathsf{HID\text{-}NIKE},\mathcal{A}}(\lambda) := \left| \Pr[\mathsf{Exp}^{\mathsf{h\text{-}adpt}}_{\mathsf{HID\text{-}NIKE},\mathcal{A}}(\lambda) \Rightarrow 1] - \frac{1}{2} \right| \leq \mathsf{negl}(\lambda),$$

for a negligible function negl.

The game $\mathsf{Exp}^{\mathsf{h\text{-}adpt}}_{\mathsf{HID\text{-}NIKE},\mathcal{A}}(\lambda)$ for the HID-NIKE $\mathsf{HID\text{-}NIKE} = (\mathsf{MasterKeyGen}, \mathsf{USKGen}, \mathsf{USKDel}, \mathsf{SharedKey})$ is identical to the game $\mathsf{Exp}^{\mathsf{adpt}}_{\mathsf{ID\text{-}NIKE},\mathcal{A}}(\lambda)$ for the corresponding ID-NIKE $\mathsf{ID\text{-}NIKE} = (\mathsf{MasterKeyGen}, \mathsf{USKGen}, \mathsf{SharedKey})$ (defined in Figure 3), except that the non-trivial winning conditions checks that $\exists \mathsf{id} \in S^\star : \mathsf{Prefix}(\mathsf{id}) \cap R_U \neq \emptyset$ instead of $\exists \mathsf{id} \in S^\star : \mathsf{id} \in R_U$ to account for the key delegation capability.

$\underline{\mathsf{MasterKeyGen}(1^\lambda)}$:
return $K \leftarrow \mathsf{KeyGen}(1^\lambda)$

$\underline{\mathsf{USKGen}(K, \mathsf{id} = (\mathsf{id}_1, \ldots, \mathsf{id}_p))}$:
for $i \in \{1, \ldots, N\}$ **do**
  $K[\![[i : \mathsf{id}]]\!] := \mathsf{Constrain}(K, [(L(i-1)+1) : \mathsf{id}_1, \ldots, (L(i-1)+p) : \mathsf{id}_p])$
return $\mathsf{usk}[\![\mathsf{id}]\!] = (K[\![[i : \mathsf{id}]]\!])_{1 \le i \le N}$

$\underline{\mathsf{SharedKey}(S, \mathsf{usk}[\![\mathsf{id}]\!])}$:
parse $(K[\![[i : \mathsf{id}]]\!])_{1 \le i \le N} := \mathsf{usk}[\![\mathsf{id}]\!]$
let $S = \{\mathsf{id}_1, \ldots, \mathsf{id}_N\}$ **such that** $\mathsf{id}_1 < \cdots < \mathsf{id}_N$
let $i \in [N]$ **such that** $\mathsf{id}_i = \mathsf{id}$
for $j \in [N]$ **do**
  parse $\mathsf{id}_{j,1}, \ldots, \mathsf{id}_{j,p_j} := \mathsf{id}_j$
  $\overline{\mathsf{id}}_j := (\mathsf{id}_{j,1}, \ldots, \mathsf{id}_{j,p_j}, \underbrace{0^\lambda, \ldots, 0^\lambda}_{\times(L-p_j)})$
return $\mathsf{Eval}(K[\![[i : \mathsf{id}]]\!], (\overline{\mathsf{id}}_1, \ldots, \overline{\mathsf{id}}_N))$

$\underline{\mathsf{USKDel}(\mathsf{usk}[\![\mathsf{id}]\!], \mathsf{id}')}$:
parse $(K[\![[i : \mathsf{id}]]\!])_{1 \le i \le N} := \mathsf{usk}[\![\mathsf{id}]\!]$
for $i \in \{1, \ldots, N\}$ **do**
  $K[\![[i : \mathsf{id}\|\mathsf{id}']]\!] := \mathsf{Constrain}(K[\![[i : \mathsf{id}]]\!], [(L(i-1)+1) : \mathsf{id}_1, \ldots, (L(i-1)+p) : \mathsf{id}_p, (L(i-1)+p+1) : \mathsf{id}'])$
return $\widehat{\mathsf{usk}}[\mathsf{id}] = (K[\![[i : \mathsf{id}\|\mathsf{id}']]\!])_{1 \le i \le N}$

Fig. 7. The transformation from a delegatable $L \cdot N$ block fixing CPRFs $\mathsf{DCPRF} = (\mathsf{KeyGen}, \mathsf{Constrain}, \mathsf{Eval})$ to a L-level N-party HID-NIKE $\mathsf{HID\text{-}NIKE}[\mathsf{DCPRF}] := (\mathsf{MasterKeyGen}, \mathsf{USKGen}, \mathsf{USKDel}, \mathsf{SharedKey})$ with identity base set $\mathcal{ID} = \{0,1\}^\lambda \setminus \{0^\lambda\}$. Identities are compared according to their lexicographic order (or any other fixed total order).

D.2 Transformation

In Figure 7 we show a transformation from a delegatable $L \cdot N$ block fixing CPRFs to a L-level N-party HID-NIKE. The transformation generalizes the transformation in Figure 7.

Theorem 16 (Delegation invariance). *If* $\mathsf{DCPRF} = (\mathsf{KeyGen}, \mathsf{Constrain}, \mathsf{Eval})$ *is a delegation invariant* $L \cdot N$ *block fixing delegatable CPRF, then the* L*-level* N*-party HID-NIKE* $\mathsf{HID\text{-}NIKE}[\mathsf{DCPRF}] = (\mathsf{MasterKeyGen}, \mathsf{USKGen}, \mathsf{USKDel}, \mathsf{SharedKey})$ *is also delegation invariant.*

Proof. Let $\mathsf{id} \in \mathcal{ID}^{\le L-1}$, $\mathsf{id}' \in \mathcal{ID}$, $\mathsf{msk} \leftarrow \mathsf{MasterKeyGen}(1^\lambda)$, $(K[\![[i : \mathsf{id}\|\mathsf{id}']]\!])_{1 \le i \le N} \leftarrow \mathsf{USKGen}(\mathsf{msk}, \mathsf{id}\|(\mathsf{id}'))$, $(K[\![[i : \mathsf{id}]]\!])_{1 \le i \le N} \leftarrow \mathsf{USKGen}(\mathsf{msk}, \mathsf{id})$, and $(K'[\![[i : \mathsf{id}\|\mathsf{id}']]\!])_{1 \le i \le N} \leftarrow \mathsf{USKDel}(\mathsf{usk}[\![\mathsf{id}]\!], \mathsf{id}')$. Since DCPRF is delegation invariant, we get for all $i \in N$ that

$$\mathsf{SD}((\mathsf{msk}, K[\![[i : \mathsf{id}]]\!], K[\![[i : \mathsf{id}\|\mathsf{id}']]\!]), (\mathsf{msk}, K[\![[i : \mathsf{id}]]\!], K'[\![[i : \mathsf{id}\|\mathsf{id}']]\!])) \le \mathsf{negl}(\lambda)$$

for a negligible function negl. This implies by Facts 2.(i) and 2.(ii)

$$\begin{aligned}&\mathsf{SD}((\mathsf{msk}, (K[\![[i' : \mathsf{id}]]\!])_{1\le i'\le N},\\ &\quad (K'[\![[i' : \mathsf{id}\|\mathsf{id}']]\!])_{1\le i'\le i-1})\|K[\![[i : \mathsf{id}\|\mathsf{id}']]\!]\|(K[\![[i' : \mathsf{id}\|\mathsf{id}']]\!])_{i+1\le i'\le N}),\\ &\qquad\qquad (\mathsf{msk}, (K[\![[i' : \mathsf{id}]]\!])_{1\le i'\le N},\\ &(K'[\![[i' : \mathsf{id}\|\mathsf{id}']]\!])_{1\le i'\le i-1})\|K'[\![[i : \mathsf{id}\|\mathsf{id}']]\!]\|(K[\![[i' : \mathsf{id}\|\mathsf{id}']]\!])_{i+1\le i'\le N}) \le \mathsf{negl}(\lambda).\end{aligned}$$

By chaining all these equations with the triangle inequality for statistical distance (Fact 2.(iii)) we get

$$\begin{aligned}&\mathsf{SD}((\mathsf{msk}, (K[\![[i : \mathsf{id}]]\!])_{1\le i\le N}, (K[\![[i : \mathsf{id}\|\mathsf{id}']]\!])_{1\le i\le N}),\\ &\qquad (\mathsf{msk}, (K[\![[i : \mathsf{id}]]\!])_{1\le i\le N}, (K'[\![[i : \mathsf{id}\|\mathsf{id}']]\!])_{1\le i\le N})) \le N\cdot\mathsf{negl}(\lambda).\end{aligned}$$

□

Theorem 17. *If* $\mathsf{DCPRF} = (\mathsf{KeyGen}, \mathsf{Constrain}, \mathsf{Eval})$ *is a correct* $L \cdot N$ *block fixing delegatable CPRF, then* $\mathsf{HID\text{-}NIKE}[\mathsf{DCPRF}]$ *is a correct* L*-level* N*-party HID-NIKE.*

The proof is analogously to Theorem 12

Theorem 18. *If* $\mathsf{DCPRF} = (\mathsf{KeyGen}, \mathsf{Constrain}, \mathsf{Eval})$ *is an adaptively pseudorandom and correct* $L \cdot N$ *block fixing delegatable CPRF, then* $\mathsf{HID\text{-}NIKE}[\mathsf{DCPRF}]$ *is an adaptively secure* L*-level* N*-party HID-NIKE. More precisely, for every adversary* $\mathcal{A}$ *there exists an adversary* $\mathcal{B}$ *with*

$$\mathsf{Adv}^{\mathsf{adpt}}_{\mathsf{ID\text{-}NIKE}[\mathsf{CPRF}],\mathcal{A}}(\lambda) \le \mathsf{Adv}^{\mathsf{adpt\text{-}pr}}_{\mathsf{CPRF},\mathcal{B}}(\lambda).$$

The proof is analogously to Theorem 13

Remark 1. A reverse transformation from HID-NIKE to delegatable, block fixing CPRF seems quite challenging to obtain, since a block-fixing CPRF has more general constrained keys and delegation capabilities than HID-NIKE. However, it is possible to formulate a CPRF variant that only supports those constrained keys and delegation capabilities used by our transformation in Figure 7. For this CPRF variant we can obtain a reverse transformation similar to Figures 4 and 6 and thus obtain the equivalence of HID-NIKE and a special CPRF variant.

E Gated identity-based encryption

In this section, we introduce gated IBE and HIBE, a variant of (hierarchical) identity-based encryption ((H)IBE) where encryption is done with a user secret key (instead of the master public key) and security of message is guaranteed as long as neither the user secret key of the sender nor the receiver is corrupted. We then show that this notion is (trivially) implied by ID-NIKE, which means that gated-IBE does not imply key agreement. Consequently, gated IBE is weaker or incomparable to PKE and strictly weaker than IBE (in a black-box sense).

E.1 Definition

Definition 23 (Gated HIBE). *A* gated hierarchical identity-based encryption *(gated HIBE)* gHIBE *with identity base set* $\mathcal{ID}$*, message space* $\mathcal{M}$ *and ciphertext space* $\mathcal{C}$ *consists of five polynomial-time algorithms* $\mathsf{gHIBE} := (\mathsf{MasterKeyGen}, \mathsf{USKGen}, \mathsf{USKDel}, \mathsf{Enc}, \mathsf{Dec})$ *with the following properties:*

- $\mathsf{MasterKeyGen}(1^\lambda, 1^L)$ *takes the unary encoded security parameter* λ *and the maximum hierarchy depth* L*. It outputs a master secret key* msk*.*
- $\mathsf{USKGen}(\mathsf{msk}, \mathsf{id} \in \mathcal{ID}^{\leq L})$ *takes a master secret key* msk *and an identity* id*. It outputs a user secret key* $\mathsf{usk}[\![\mathsf{id}]\!]$ *for an identity* $\mathsf{id} \in \mathcal{ID}^{\leq L}$*. We assume that one can efficiently derive* id *from* $\mathsf{usk}[\![\mathsf{id}]\!]$*.*
- $\mathsf{USKDel}(\mathsf{usk}[\![\mathsf{id}]\!], \mathsf{id}' \in \mathcal{ID})$ *takes a user secret key* $\mathsf{usk}[\![\mathsf{id}]\!]$ *for* $\mathsf{id} \in \mathcal{ID}^{\leq L-1}$ *and an additional identity component* id'*. It outputs a user secret* $\mathsf{usk}[\![\mathsf{id}\|(\mathsf{id}')]\!]$ *for* $\mathsf{id}\|(\mathsf{id}')$*.*
- $\mathsf{Enc}(\mathsf{usk}[\![\mathsf{id}_S]\!], \mathsf{id}_R \in \mathcal{ID}^{\leq L}, m \in \mathcal{M})$ *taks a* user secret key $\mathsf{usk}[\![\mathsf{id}_S]\!]$ *(the sender's user secret key), an identity* id_R *(the receiver identity), and a message* m *and returns a ciphertext* $\mathsf{ct} \in \mathcal{C}$ *that encrypts the message* m *with respect to the identity* id'*.*
- $\mathsf{Dec}(\mathsf{usk}[\![\mathsf{id}_R]\!], \mathsf{ct} \in \mathcal{C})$ *is deterministic (the other algorithms may be probabilistic) and takes a user secret key* $\mathsf{usk}[\![\mathsf{id}_R]\!]$ *and a ciphertext* ct*. It returns a message* $m \in \mathcal{M}$ *or the reject symbol* $\perp$*.*

The definition of gated IBE is obtained by removing the argument 1^L from the $\mathsf{MasterKeyGen}$ algorithm and using $L = 1$ instead (and forbidding the usage of the identity ε). Also, the USKDel algorithm is omitted, because it is useless in this setting.

Definition 24 (Delegation invariance). *A gated HIBE* $\mathsf{gHIBE} = (\mathsf{MasterKeyGen}, \mathsf{USKGen}, \mathsf{USKDel}, \mathsf{Enc}, \mathsf{Dec})$ *is* delegation invariant *iff,*

- $\forall \lambda, L, \in \mathbb{N}_+, \mathsf{id} \in \mathcal{ID}^{\leq L-1}, \mathsf{id}' \in \mathcal{ID}$*,*
- *for* $\mathsf{msk} \leftarrow \mathsf{MasterKeyGen}(1^\lambda, 1^L)$*,*
- $\mathsf{usk}[\![\mathsf{id}]\!] \leftarrow \mathsf{USKGen}(\mathsf{msk}, \mathsf{id})$*,*
- $\mathsf{usk}[\![\mathsf{id}\|(\mathsf{id}')]\!] \leftarrow \mathsf{USKDel}(\mathsf{usk}[\![\mathsf{id}]\!], \mathsf{id}')$*, and*
- $\widehat{\mathsf{usk}}[\mathsf{id}\|(\mathsf{id}')] \leftarrow \mathsf{USKGen}(\mathsf{msk}, \mathsf{id}\|(\mathsf{id}'))$*, we have*

$$(\mathsf{msk}, \mathsf{usk}[\![\mathsf{id}]\!], \mathsf{usk}[\![\mathsf{id}\|(\mathsf{id}')]\!]) \approx_s (\mathsf{msk}, \mathsf{usk}[\![\mathsf{id}]\!], \widehat{\mathsf{usk}}[\mathsf{id}\|(\mathsf{id}')]),$$

where the probability is taken over the randomness used to sample msk*,* $\mathsf{usk}[\![\mathsf{id}]\!]$*,* $\mathsf{usk}[\![\mathsf{id}\|(\mathsf{id}')]\!]$*, and* $\widehat{\mathsf{usk}}[\mathsf{id}\|(\mathsf{id}')]$*. If the gated HIBE scheme is defined relative to an oracle, we need*

$$(\mathsf{mpk}, \mathsf{msk}, \mathcal{O}, \mathsf{usk}[\![\mathsf{id}]\!], \mathsf{usk}[\![\mathsf{id}\|(\mathsf{id}')]\!]) \approx_s (\mathsf{mpk}, \mathsf{msk}, \mathcal{O}, \mathsf{usk}[\![\mathsf{id}]\!], \widehat{\mathsf{usk}}[\mathsf{id}\|(\mathsf{id}')]),$$

where $\mathcal{O}$ *is the entire function table of the oracle and all other variables are defined as above.*

Definition 25 (Correctness). *A delegation invariant gated HIBE* $\mathsf{gHIBE} = (\mathsf{MasterKeyGen}, \mathsf{USKGen}, \mathsf{USKDel}, \mathsf{Enc}, \mathsf{Dec})$ *is* correct *iff,*

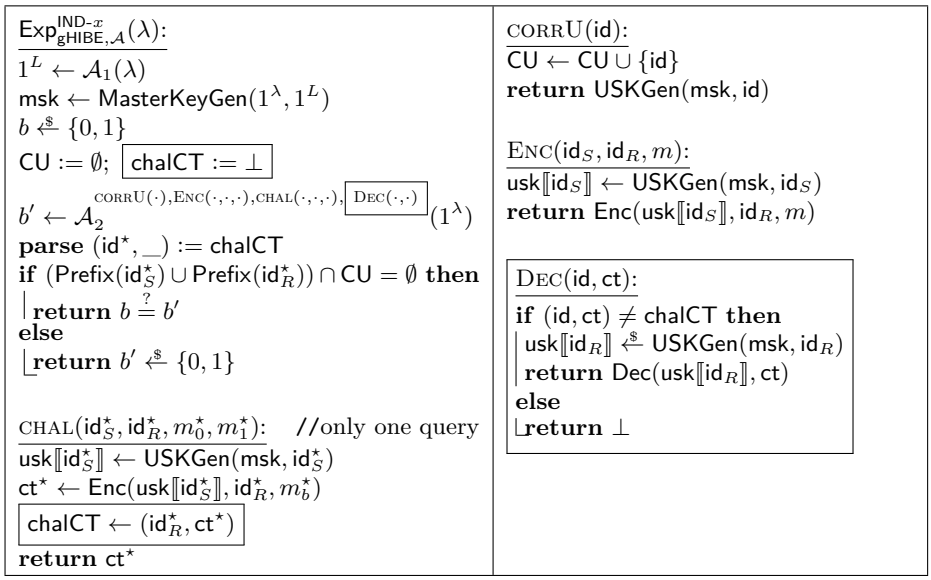

Fig. 8. The adaptive IND-CPA and adaptive $\boxed{\text{IND-CCA}}$ security experiment for a gated HIBE $\mathsf{gHIBE} = (\mathsf{MasterKeyGen}, \mathsf{USKGen}, \mathsf{USKDel}, \mathsf{Enc}, \mathsf{Dec})$.

- $\forall \lambda, L \in \mathbb{N}_+, \mathsf{id}_S, \mathsf{id}_R \in \mathcal{ID}^{\leq L}$,
- *for* $\mathsf{msk} \leftarrow \mathsf{MasterKeyGen}(1^\lambda, 1^L)$,
- $\mathsf{usk}[\![\mathsf{id}_S]\!] \leftarrow \mathsf{USKGen}(\mathsf{msk}, \mathsf{id}_S)$, *and*
- $\mathsf{usk}[\![\mathsf{id}_R]\!] \leftarrow \mathsf{USKGen}(\mathsf{msk}, \mathsf{id}_R)$, *we have*

$$\Pr[\mathsf{Dec}(\mathsf{usk}[\![\mathsf{id}_R]\!], \mathsf{Enc}(\mathsf{usk}[\![\mathsf{id}_S]\!], \mathsf{id}_R, m)) = m] \geq 1 - \mathsf{negl}(\lambda)$$

for a negligible function negl*, where the probability is taken over the randomness used to sample* msk*,* $\mathsf{usk}[\![\mathsf{id}]\!]$*, and the randomness of* Enc *and* Dec*. If* $\mathsf{negl} = 0$*, we call* gHIBE *perfectly correct.*

Definition 26 (Security). *A gated HIBE* $\mathsf{gHIBE} := (\mathsf{MasterKeyGen}, \mathsf{USKGen}, \mathsf{USKDel}, \mathsf{Enc}, \mathsf{Dec})$ *scheme is adaptively IND-x secure with* $x \in \{CPA, CCA\}$ *when for every PPT adversary* $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$

$$\mathsf{Adv}^{\mathsf{IND}\text{-}x}_{\mathsf{gHIBE},\mathcal{A}}(\lambda) := 2\left|\Pr[\mathsf{Exp}^{\mathsf{IND}\text{-}x}_{\mathsf{gHIBE},\mathcal{A}}(\lambda) \Rightarrow 1] - \frac{1}{2}\right| \leq \mathsf{negl}(\lambda)$$

for a negligible function negl*. The experiment* $\mathsf{Exp}^{\mathsf{IND}\text{-}x}_{\mathsf{gHIBE},\mathcal{A}}(\lambda)$ *is defined in Figure 8.*

Correctness and security for gated IBE are identical to the definitions for delegation invariant gated HIBE.

E.2 SKE

We recall the standard definition of secret key encryption.

$\underline{\mathsf{Exp}^{\text{IND-CPA}}_{\mathsf{SKE}=(\mathsf{Enc},\mathsf{Dec}),\mathcal{A}}(\lambda)}$:	$\underline{\textsc{Enc}(m)}$:
$k \xleftarrow{\$} \mathcal{K}$	**return** $\mathsf{Enc}(k, m)$
$b \xleftarrow{\$} \{0,1\}$	
$b' \leftarrow \mathcal{A}^{\textsc{Chal}(\cdot,\cdot),\textsc{Enc}(\cdot)}(1^\lambda)$	$\underline{\textsc{Chal}(m_0 \in \mathcal{M}, m_1 \in \mathcal{M})}$: //only one query
return $b \stackrel{?}{=} b'$	**return** $\mathsf{Enc}(k, m_b)$
	$\boxed{\underline{\textsc{Dec}(\mathsf{ct})}: \\ \textbf{return } \mathsf{Dec}(k, \mathsf{ct})}$

Fig. 9. The IND-CPA and $\boxed{\text{IND-CCA}}$ security game for SKE.

Definition 27 (SKE). *A* secret key encryption (SKE) *scheme* SKE *for key space* $\mathcal{K}$ *message space* $\mathcal{M}$ *with ciphertext space* $\mathcal{C}$ *consist of the following two probabilistic algorithms:*

$\mathsf{Enc}(k, m)$**:** *Given a key* k *and a message* $m \in \mathcal{M}$ *as input, it outputs a ciphertext* $\mathsf{ct} \in \mathcal{C}$*.*

$\mathsf{Dec}(k, \mathsf{ct})$**:** *Given a key* k *and a ciphertext* $\mathsf{ct} \in \mathcal{C}$ *as input, it outputs a message* $m \in \mathcal{M}$ *or* $\perp$ *(indicating a failure).*

Definition 28. *An SKE* $\mathsf{SKE} = (\mathsf{Enc}, \mathsf{Dec})$ *is correct iff for every message* $m \in \mathcal{M}$ *we have*

$$\Pr[\mathsf{Dec}(k, \mathsf{Enc}(k, m)) = m] \geq 1 - \mathsf{negl}(\lambda)$$

for a negligible function negl*, where the probability is taken over sampling* $k \leftarrow \mathcal{K}$ *and the randomness of* Enc *and* Dec*.*

Definition 29. *An SKE is IND-x secure with* $x \in \{CPA, CCA\}$ *secure iff for every PPT adversary* $\mathcal{A}$

$$\mathsf{Adv}^{\text{IND-}x}_{\mathsf{SKE},\mathcal{A}}(\lambda) := 2\left|\Pr[\mathsf{Exp}^{\text{IND-}x}_{\mathsf{SKE},\mathcal{A}}(\lambda) \Rightarrow 1] - \frac{1}{2}\right| \leq \mathsf{negl}(\lambda)$$

for a negligible function negl*. The experiment* $\mathsf{Exp}^{\text{IND-}x}_{\mathsf{SKE},\mathcal{A}}(\lambda)$ *is defined in Figure 9.*

E.3 Transformation

In Figure 10 we show a transformation from a (2-party) HID-NIKE to an (IND-CPA secure) gated HIBE scheme. The transformation preserves the hierarchy depth, and, in particular, it can be used to transform an ID-NIKE to an (IND-CPA secure) gated IBE. The transformation also makes use of a (IND-CPA secure) SKE scheme. It is well known that SKE can be built from one-way functions (OWFs) and since we can trivially get OWFs from (H)ID-NIKE, this is not an additional assumption.

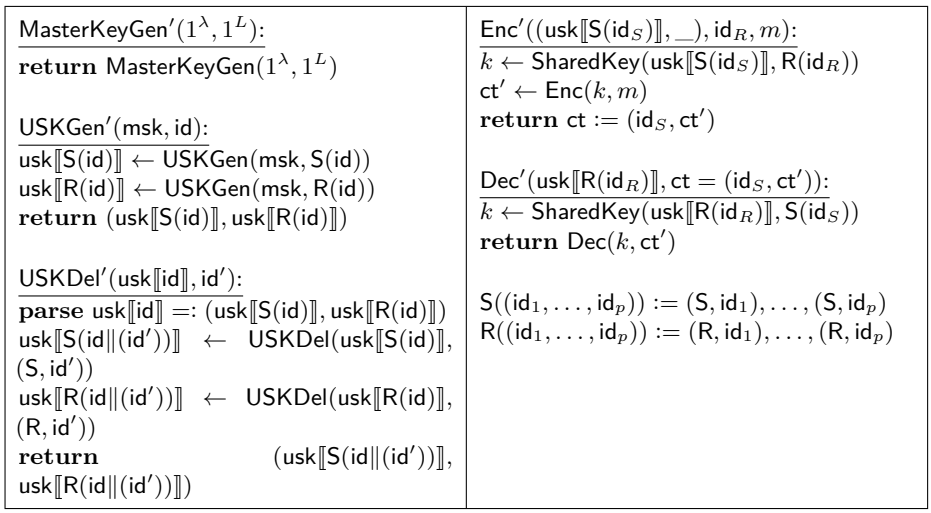

Fig. 10. The transformation from a (2-party) HID-NIKE $\mathsf{HID\text{-}NIKE} = (\mathsf{MasterKeyGen}, \mathsf{USKGen}, \mathsf{USKDel}, \mathsf{SharedKey})$ with identity base set $\{\mathsf{S}, \mathsf{R}\} \times \mathcal{ID}$ and shared key space $\mathcal{K}$ and an SKE $\mathsf{SKE} = (\mathsf{Enc}, \mathsf{Dec})$ with the same key space $\mathcal{K}$, message space $\mathcal{M}$, and ciphertext space $\mathcal{C}$ to a gated HIBE scheme $\mathsf{gHIBE}[\mathsf{HID\text{-}NIKE}, \mathsf{SKE}] = (\mathsf{MasterKeyGen}', \mathsf{USKGen}', \mathsf{USKDel}', \mathsf{Enc}', \mathsf{Dec}')$ with identity base set $\mathcal{ID}$ and message space $\mathcal{M}$.

Theorem 19. *If* HID-NIKE *is a delegation invariant HID-NIKE, then* gHIBE[HID-NIKE, SKE] *is delegation invariant gated HIBE.*

Proof. The algorithms for user secret key generation and delegation for gHIBE[HID-NIKE, SKE] run the corresponding algorithms for HID-NIKE twice on their respective inputs and return both outputs. Thus, the delegation invariance of HID-NIKE trivially carries over to gHIBE[HID-NIKE, SKE]. □

Theorem 20. *If* HID-NIKE *is a correct HID-NIKE and* SKE *is a correct SKE, then* gHIBE[HID-NIKE, SKE] *is a correct gated HIBE.*

Proof. Let $L \in \mathbb{N}_+$, $\mathsf{msk} \xleftarrow{\$} \mathsf{MasterKeyGen}(1^\lambda, 1^L)$, $\mathsf{id}_S, \mathsf{id}_R \in \mathcal{ID}^{\leq L}$, $(\mathsf{usk}[\![\mathsf{S}(\mathsf{id}_S)]\!], \_) \leftarrow \mathsf{USKGen}(\mathsf{msk}, \mathsf{id}_S)$, and $(\_, \mathsf{usk}[\![\mathsf{R}(\mathsf{id}_R)]\!]) \leftarrow \mathsf{USKGen}(\mathsf{msk}, \mathsf{id}_R)$. Then with overwhelming probability $\mathsf{SharedKey}(\mathsf{usk}[\![\mathsf{S}(\mathsf{id}_S)]\!], \mathsf{R}(\mathsf{id}_R)) = \mathsf{SharedKey}(\mathsf{usk}[\![\mathsf{R}(\mathsf{id}_R)]\!], \mathsf{S}(\mathsf{id}_S)) =: k$ by correctness of HID-NIKE. Similar, with overwhelming probability $\mathsf{Dec}(k, \mathsf{Enc}(k, m)) = m$ by correctness of SKE. Combining these two statements shows correctness of gHIBE[HID-NIKE, SKE]. □

Theorem 21. *If* HID-NIKE *is adaptively secure and* SKE *is IND-x secure (with* $x \in \{CPA, CCA\}$*), then* gHIBE[HID-NIKE, SKE] *is adaptively IND-x secure.*

Proof. The proof proceeds via a simple hybrid argument. The first hybrid G_0 is the real adaptive IND-CPA game for gHIBE[HID-NIKE, SKE].

Let $q_{E/D}$ be the number of encryption and decryption queries the adversary $\mathcal{A}$ does before the CHAL query. We say that an identity appears as sender/receiver identity at position $i \in [q_{E/D} + 1]$ if it is used as sender/receiver identity in the i-th ENC/DEC query[5] or if it appears in no ENC/DEC query before the CHAL query and $i = q_{E/D} + 1$.

In the next hybrid G_1, the game guesses two indices $i^\star, j^\star \xleftarrow{\$} [q_{E/D} + 1]$. If the challenge sender identity $\mathsf{id}_S^\star$ appears as sender identity in an ENC/DEC query for the first time at position $i^\star$ and the challenge receiver identity $\mathsf{id}_R^\star$ appears as receiver identity in an ENC/DEC query for the first time at position $j^\star$, the game proceeds as normal. Otherwise, the game aborts and outputs a random bit.

Lemma 3 ($\mathsf{G}_0 \rightsquigarrow \mathsf{G}_1$)**.**

$$\Pr[\mathsf{G}_0^{\mathcal{A}} \Rightarrow 1] = \left(\frac{1}{q_{E/D} + 1}\right)^2 \Pr[\mathsf{G}_1^{\mathcal{A}} \Rightarrow 1]$$

Proof. The indices $i^\star$ and $j^\star$ are information-theoretically hidden from the adversary. Thus, each index will be the correct guess with probability $1/(q_{E/D}+1)$ □

In the following, we use $\mathsf{id}_S^\star$ to denote the sender identity at position $i^\star$ and $\mathsf{id}_R^\star$ to denote the receiver identity at position $j^\star$. Note that this is not a real variable name conflict with the identities in the CHAL query, because the game only proceeds if these identities match $\mathsf{id}_S^\star$ and $\mathsf{id}_R^\star$.

[5] In DEC the sender's identity is the identity that is part of the ciphertext and the receiver identity is the identity used to generate the user secret key for decryption.

The final hybrid is G_2, which proceeds identically to G_0 except that in the CHAL query the real HID-NIKE shared key $k^\star \leftarrow \mathsf{SharedKey}(\mathsf{usk}[\![\mathsf{id}_S^\star]\!], \mathsf{R}(\mathsf{id}_R^\star))$ is replaced by a uniformly random shared key $k^\star \xleftarrow{\$} \mathcal{K}$. Moreover, all $\textsc{Enc}(\mathsf{id}_S^\star, \mathsf{id}_R^\star, m)$ queries are answered with $(\mathsf{id}_S^\star, \mathsf{ct}', \tau)$ where $\mathsf{ct}' \leftarrow \mathsf{Enc}(k^\star, m)$ (instead of using the keys derived via $\mathsf{SharedKey}(\mathsf{usk}[\![\mathsf{id}_S^\star]\!], \mathsf{R}(\mathsf{id}_R^\star))$). Finally, all $\textsc{Dec}(\mathsf{id}_R^\star, \mathsf{ct} = (\mathsf{id}_S^\star, \mathsf{ct}'))$ queries are answered by returning $\mathsf{Dec}(k^\star, \mathsf{ct}')$ (instead of using the keys derived via $\mathsf{SharedKey}(\mathsf{usk}[\![\mathsf{id}_R^\star]\!], \mathsf{S}(\mathsf{id}_S^\star))$).

Lemma 4 ($\mathsf{G}_1 \rightsquigarrow \mathsf{G}_2$). *For every adversary $\mathcal{A}$ there exists an adversary $\mathcal{B}$ with*

$$\left|\Pr[\mathsf{G}_1^{\mathcal{A}} \Rightarrow 1] - \Pr[\mathsf{G}_2^{\mathcal{A}} \Rightarrow 1]\right| \leq \mathsf{Adv}^{\mathsf{h\text{-}adpt}}_{\mathsf{HID\text{-}NIKE},\mathcal{A}}(\lambda).$$

Proof. The first stage of the reduction just forwards 1^L. The second stage of the reduction then samples initially a random bit $b \xleftarrow{\$} \{0,1\}$. It answers all CORRU query from the adversary in the adaptive IND-CPA security game for $\mathsf{gHIBE}[\mathsf{HID\text{-}NIKE}, \mathsf{SKE}]$ by making a CORRU query in the adaptive security experiment for HID-NIKE and forwarding the result.

As soon as the reduction has learned the two challenge identities $\mathsf{id}_S^\star$, $\mathsf{id}_R^\star$, which can either happen when the adversary has made ENC/DEC queries with positions $i^\star$ and $j^\star$ or the CHAL query, the reduction makes a $\textsc{Chal}(\mathsf{id}_S^\star, \mathsf{id}_R^\star)$ to obtain a candidate shared key $k^\star$ and stores it. Each $\textsc{Enc}(\mathsf{id}_S, \mathsf{id}_R, m)$ query is answered as follows: If $(\mathsf{id}_S, \mathsf{id}_R) = (\mathsf{id}_S^\star, \mathsf{id}_R^\star)$, the reduction uses the key $k^\star$ to answer the query (i.e. it returns $(\mathsf{id}_S, \mathsf{Enc}(k^\star, m))$). Otherwise, the reduction makes a reveal query $k \leftarrow \textsc{Reveal}(\{\mathsf{S}(\mathsf{id}_S), \mathsf{R}(\mathsf{id}_R)\}, \mathsf{S}(\mathsf{id}_S))$ and uses the key k to answer the query (i.e. it returns $(\mathsf{id}_S, \mathsf{Enc}(k, m))$).

Each $\textsc{Dec}(\mathsf{id}_R, \mathsf{ct} = (\mathsf{id}_S, \mathsf{ct}'))$ query is answered as follows: If $(\mathsf{id}_S, \mathsf{id}_R) = (\mathsf{id}_S^\star, \mathsf{id}_R^\star)$, the reduction uses the key $k^\star$ to answer the query (i.e. returns $\mathsf{Dec}(k^\star, \mathsf{ct}')$). Otherwise, the reduction makes a reveal query $k \leftarrow \textsc{Reveal}(\{\mathsf{S}(\mathsf{id}_S), \mathsf{R}(\mathsf{id}_R)\}, \mathsf{S}(\mathsf{id}_S))$ and uses the key k to answer the query (i.e. returns $\mathsf{Dec}(k, \mathsf{ct}')$).

In the $\textsc{Chal}(\mathsf{id}_S^\star, \mathsf{id}_R^\star, m_0^\star, m_1^\star)$ query, the reduction returns $(\mathsf{id}_S, \mathsf{Enc}(k^\star, m_b))$. If the adversary outputs in the end b', the reduction outputs $b = b'$.

Assume that the guess $i^\star$, $j^\star$ for the positions of the challenge identities was correct. Otherwise, the reduction and the games G_1 and G_2 abort. If the key $k^\star$ obtained in the CHAL query is the real shared key $\mathsf{SharedKey}(\mathsf{usk}[\![\mathsf{S}(\mathsf{id}_S^\star)]\!], \mathsf{R}(\mathsf{id}_R^\star))$, the reduction perfectly simulates G_0 and if $k^\star$ is uniformly random, the reduction perfectly simulates G_2.

The reduction never makes a REVEAL query for the user set $\{\mathsf{S}(\mathsf{id}_S^\star, \mathsf{id}_R^\star)\}$. The reduction only asks for the user secret key of one of the challenge identities if the adversary does so. □

Lemma 5 (G_2). *For every adversary $\mathcal{A}$ there exists an adversary $\mathcal{B}$ with*

$$\Pr[\mathsf{G}_2^{\mathcal{A}} \Rightarrow 1] \leq \mathsf{Adv}^{\mathsf{IND\text{-}CPA}}_{\mathsf{SKE},\mathcal{A}}(\lambda)$$

Proof. The reduction is straightforward. The reduction gets 1^L and samples initially a master secret key $\mathsf{msk} \leftarrow \mathsf{MasterKeyGen}(1^\lambda, 1^L)$. It uses msk to answer all CORRU queries honestly.

For all $\textsc{Enc}(\mathsf{id}_S, \mathsf{id}_R, m)$ queries that happen before both $\mathsf{id}_S^\star$ and $\mathsf{id}_R^\star$ are known or with $(\mathsf{id}_S, \mathsf{id}_R) \neq (\mathsf{id}_S^\star, \mathsf{id}_R^\star)$ are answered by returning $\mathsf{Enc}(\mathsf{SharedKey}(\mathsf{usk}[\![\mathsf{S}(\mathsf{id}_S)]\!], \mathsf{R}(\mathsf{id}_R)), m)$. Each $\textsc{Enc}(\mathsf{id}_S, \mathsf{id}_R, m)$ query with $(\mathsf{id}_S, \mathsf{id}_R) = (\mathsf{id}_S^\star, \mathsf{id}_R^\star)$ is answered by making an $\textsc{Enc}(m)$ query to the IND-x security game to get ct and returning $(\mathsf{id}_S, \mathsf{ct})$.

For all $\textsc{Dec}(\mathsf{id}_R, \mathsf{ct} = (\mathsf{id}_S, \mathsf{ct}'))$ queries that happen before $\mathsf{id}_S^\star$ and $\mathsf{id}_R^\star$ are known or with $(\mathsf{id}_S, \mathsf{id}_R) \neq (\mathsf{id}_S^\star, \mathsf{id}_R^\star)$ are answered by returning $\mathsf{Dec}(\mathsf{SharedKey}(\mathsf{usk}[\![\mathsf{R}(\mathsf{id}_R)]\!], \mathsf{S}(\mathsf{id}_S)), \mathsf{ct}')$. Each $\textsc{Dec}(\mathsf{R}(\mathsf{id}_R), \mathsf{ct} = (\mathsf{S}(\mathsf{id}_S), \mathsf{ct}'))$ query with $(\mathsf{id}_S, \mathsf{id}_R) = (\mathsf{id}_S^\star, \mathsf{id}_R^\star)$ is answered by making a $\textsc{Dec}(\mathsf{ct}')$ query to the IND-CCA security game to get and return m.

In the $\textsc{chal}(\mathsf{id}_S^\star, \mathsf{id}_R^\star, m_0^\star, m_1^\star)$ query, the reduction makes a $\textsc{Chal}(m_0^\star, m_1^\star)$ query to get $\mathsf{ct}^\star$ and returns $(\mathsf{id}_S^\star, \mathsf{ct}^\star)$. Finally, the reduction returns the same bit as the adversary.

Assume that the guess $i^\star$, $j^\star$ for the positions of the challenge identities was correct. Otherwise, the reduction and the game G_2 abort. It is easy to see that the reduction perfectly simulates the game G_2 with the same challenge bit as used in the IND-CPA security game for SKE. □

Combining Lemmata 3-5 yields Theorem 21. □

Avoiding the length-doubling of user secret keys. The transformation above doubles the length of the user secret keys of the (H)ID-NIKE, because each user uses different identities and user secret keys for encryption and decryption. When we use the same identity and user secret key for encryption and decryption, we can still argue that the resulting gated (H)IBE is IND-CPA secure.

The reduction uses in hybrid G_2 the key $k^\star$ for both $\textsc{Enc}(\mathsf{id}_S^\star, \mathsf{id}_R^\star, m)$ (as the reduction presented here) and for $\textsc{Enc}(\mathsf{id}_R^\star, \mathsf{id}_S^\star, m)$ queries. Thus, we also need to assume that the underlying (H)ID-NIKE has a negligible correctness error here.

However, this variant of the transformation does not achieve IND-CCA security because of the following attack: An attacker receiving the challenge ciphertext $(\mathsf{id}_S^\star, \mathsf{ct}^\star)$ for receiver $\mathsf{id}_R^\star$ can make a $\textsc{Dec}(\mathsf{id}_S^\star, (\mathsf{id}_R^\star, \mathsf{ct}^\star))$ query to retrieve the challenge message and win the game (note that this query is allowed by the IND-CCA game, only $\textsc{Dec}(\mathsf{id}_R^\star, (\mathsf{id}_S^\star, \mathsf{ct}^\star))$ is forbidden). Interestingly, the Encrypt-then-MAC paradigm [5] (splitting the (H)ID-NIKE key into an SKE and a MAC part, adding a MAC tag over id_S and ct to each ciphertext, and verifying the MAC tag before decryption) to achieve CCA security for SKE does not seem to help here, because of the following circularity issue. To rely on the unforgeability of the MAC, we first need to apply (H)ID-NIKE security to make the MAC key uniformly random. However, to apply (H)ID-NIKE security we have to argue that a $\textsc{Dec}(\mathsf{id}_S^\star, (\mathsf{id}_R^\star, \mathsf{ct}^\star, \tau^\star))$ query can be answered with $\perp$ safely, which requires the unforgeability of the MAC. Thus, we do not know how to construct IND-CCA secure gated (H)IBE from (H)ID-NIKE without doubling the user secret key length.

From traditional (H)IBE to gated (H)IBE. It is trivial that traditional (H)IBE implies gated (H)IBE. The transformation just has to store the master public key

mpk of the (H)IBE in each user secret key of the gated (H)IBE. This transformation clearly preserves the number of levels, delegation invariance and correctness. For encryption, mpk is used for encryption and the rest of the user secret key is ignored. The reduction can simulate the Enc oracle by using mpk from the underlying HIBE scheme. All other oracles in the (H)IBE have a direct counterpart in the gated (H)IBE security experiment. Thus the reduction preserves all standard security notions (adaptive/selective IND-CPA/CCA security).

F History of changes

Version 5 (2026-01-29)
- The previous idea to extend the proof by [17] to non-perfect correctness was flawed and got removed. The new result only rules out perfectly correct key agreement from one-way functions.
- A related problem arose with having a non-injective $\mathcal{O}_{\mathsf{label}}$ oracle. This has been resolved by using an injective $\mathcal{O}_{\mathsf{ilabel}}$ that can only be accessed indirectly via a $\mathcal{O}_{\mathsf{same}}$ oracle (see the end of the new technical overview for details).

Version 4 (2025-10-03)
- A new proof strategy to extend the proof by [17] to key agreement with non-perfect correctness is used. The previous proof strategy (specifically, the argument that good rounds are well distributed) is flawed, which was pointed out by Noam Mazor.

Version 3 (2025-08-25):
- Increased the number of rounds of Eve and Eve outputs the shared key from a random round (instead of picking a random shared key from the set of keys of all rounds), so that keys that Eve sees more often are more likely to be output. With these changes, we can argue that Eve achieves constant advantage instead of inverse polynomial advantage, at the cost of only doubling the number of queries Eve makes (which is better than generic amplification).
- A previous remark that the inverse polynomial advantage does not appear when we consider indistinguishability-based instead of one-way security for key agreement (and thus our result suffices to show optimality of Merkle puzzles) was wrong. This remark was removed. The new version shows optimality of Merkle puzzles directly, even for one-way security.
- Added a few more references on constrained PRFs.
- Fixed a few typos and editiorial improvements.

Version 2 (2025-07-24):
- Fixed a typo.